

Data-Driven Approximation of Earth–Mars Pork-Chop Surfaces

Scott Nguyen

Dept. of Electrical & Computer Engineering
University of New Mexico

ECE 506 Final Project Presentation

Motivation & Problem Statement

- Interplanetary trajectory design requires repeated evaluation of transfer costs over large launch windows.
- These trade spaces are visualized using **pork-chop plots**:

$$\Delta V(\text{departure date, time-of-flight})$$

- Classical workflow:
 - Solve Lambert's problem at every grid point.
 - Becomes computationally expensive for wide windows or repeated optimization.
- **Project objective:**
 - Train a neural-network surrogate to approximate the Earth–Mars pork-chop surface.
 - Enable fast evaluation, grid search, and optimization using the surrogate.

Background & Related Work

- **RL for robust low-thrust trajectories:** Zavoli & Federici formulate Earth–Mars low-thrust transfers as Markov decision processes and train deep RL guidance and control networks that are robust to process noise, navigation uncertainty, and missed-thrust events.
- **Meta-RL for multiple uncertainties:** Federici & Zavoli extend this to meta-reinforcement learning with recurrent networks, learning a closed-loop policy that adapts across environments with mixed dynamic and control uncertainties for a fuel-optimal Earth–Mars transfer.
- **Surrogate NNs for trajectory planning:** Ruff et al. use neural-network surrogates inside a simulation-based optimization loop to design hypersonic approach-and-landing reference trajectories, showing large reductions in computation time versus direct simulation-based tuning.
- This project:
 - Adopts the idea of NN surrogates from Ruff et al., but in a simpler supervised setting.
 - Applies it to an impulsive Earth–Mars Lambert pork-chop rather than low-thrust optimal control.

Objective Function (What We Minimize)

We solve a two-impulse heliocentric transfer with Lambert's problem (Earth departure, Mars arrival).

Decision variables:

$$\mathbf{x} \triangleq \begin{bmatrix} t_{\text{dep}} \\ \text{TOF} \end{bmatrix}, \quad t_{\text{arr}} = t_{\text{dep}} + \text{TOF}$$

Lambert-based total cost:

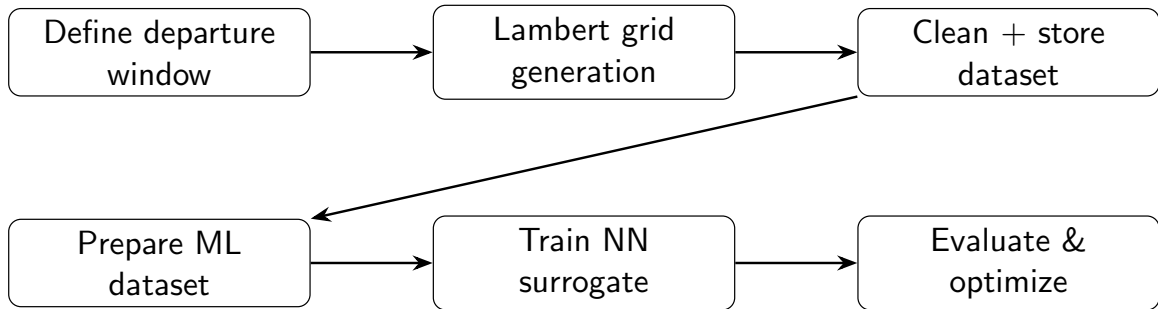
$$J(t_{\text{dep}}, \text{TOF}) = \Delta V_{\text{tot}} = \|\mathbf{v}_{\text{dep}}^* - \mathbf{v}_E(t_{\text{dep}})\| + \|\mathbf{v}_M(t_{\text{arr}}) - \mathbf{v}_{\text{arr}}^*\|$$

where $(\mathbf{v}_{\text{dep}}^*, \mathbf{v}_{\text{arr}}^*)$ are the Lambert transfer velocities solving

$$(\mathbf{v}_{\text{dep}}^*, \mathbf{v}_{\text{arr}}^*) = \text{Lambert}(\mathbf{r}_E(t_{\text{dep}}), \mathbf{r}_M(t_{\text{arr}}), \text{TOF}).$$

Feasibility / filtering: solutions with $\Delta V_{\text{tot}} > 20 \text{ km/s}$ are discarded (treated as invalid / NaN).

Workflow Overview



Methods & How We Compare

1) Ground-truth surface (Lambert grid):

- Sample t_{dep} daily over a 2-year window (~ 731 points).
- Sample TOF over 60 values within the derived feasible arrival bounds.
- Evaluate $J(t_{\text{dep}}, \text{TOF})$ at each grid point $\Rightarrow$ pork-chop surface.

2) “True objective” optimization (continuous):

- Optimize the same Lambert objective $J(t_{\text{dep}}, \text{TOF})$ over continuous bounds.
- Differential Evolution (DE) used to reduce sensitivity to nonconvexity / invalid regions.
- Output: best $(t_{\text{dep}}, \text{TOF})$ and optimization history (path over pork-chop).

3) Surrogate model (NN regression):

- Train $\hat{J}(t_{\text{dep}}, \text{TOF})$ on valid grid samples only.
- Evaluate: MAE/RMSE between $\hat{J}$ and J on held-out data.

4) Surrogate-based optimization (continuous):

- Minimize $\hat{J}(t_{\text{dep}}, \text{TOF})$ with DE.

Dataset Generation

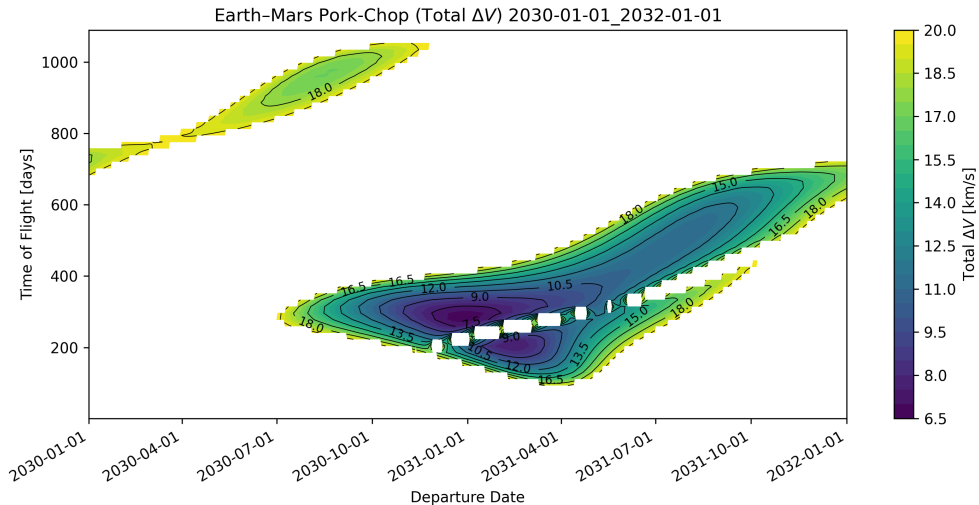
- Earth–Mars Lambert transfers using poliastro.
- Departure dates sampled daily over 2-year windows.
- Time-of-flight sampled uniformly within feasible arrival bounds (derived from arrival window).
- Invalid solutions ($\Delta V > 20$ km/s) removed.
- Multiple windows evaluated:

2030–2032, 2032–2034, 2034–2036, 2036–2038, 2038–2040

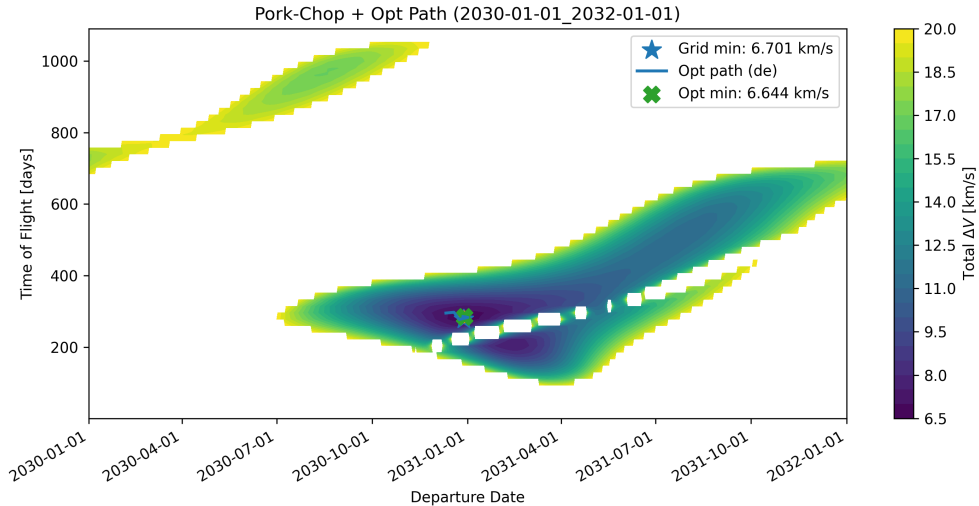
Run Summary (All Windows)

Window	Grid min	True opt min	Surrogate MAE	Surrogate RMSE
2030–2032	6.7009	6.6445	0.7313	1.2269
2032–2034	6.1420	6.1211	0.7985	1.3743
2034–2036	5.5770	5.5707	1.5362	3.0171
2036–2038	6.6032	6.6002	1.5831	3.1374
2038–2040	5.7429	5.7125	1.4953	2.5518

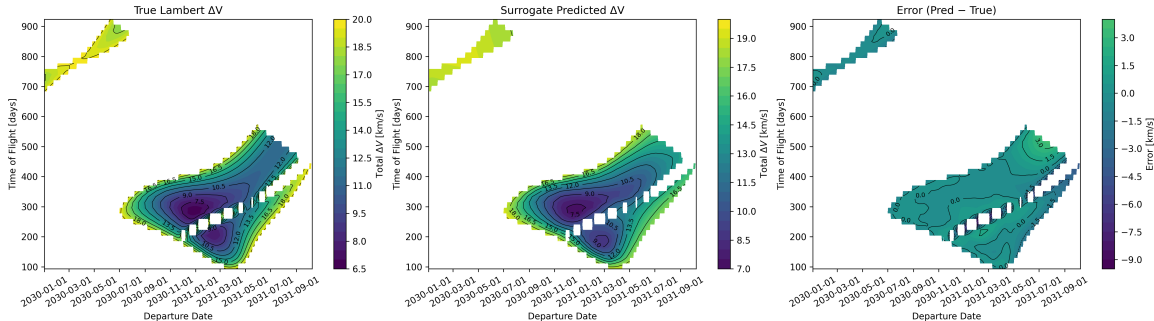
Baseline Pork-Chop Surface (Lambert)



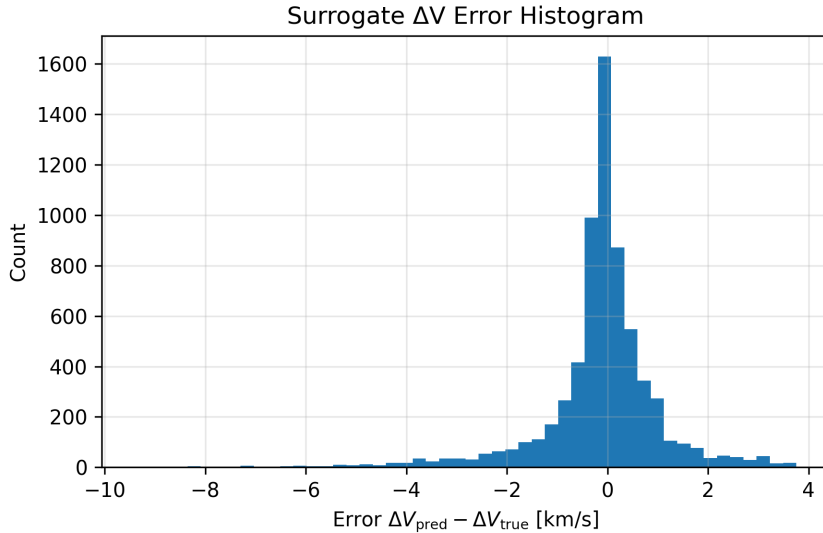
True Objective Optimization



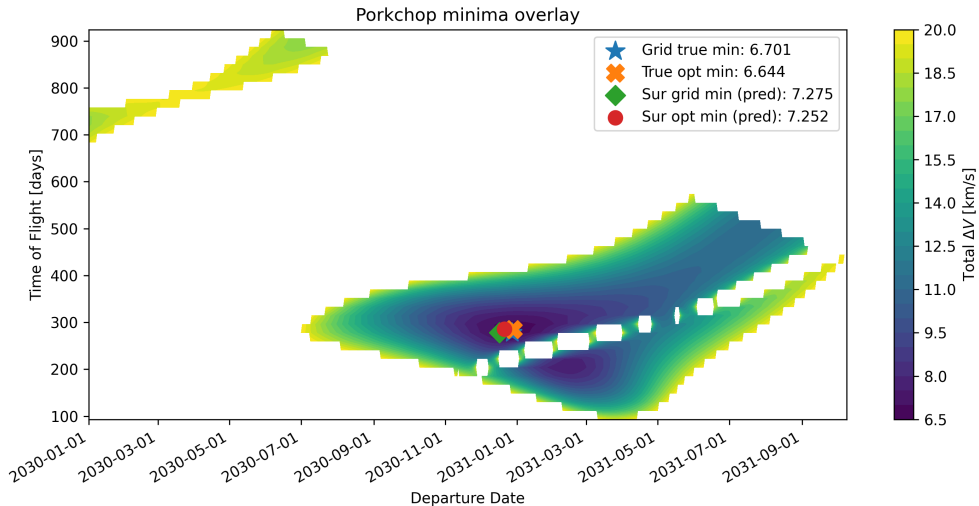
Surrogate Model Evaluation



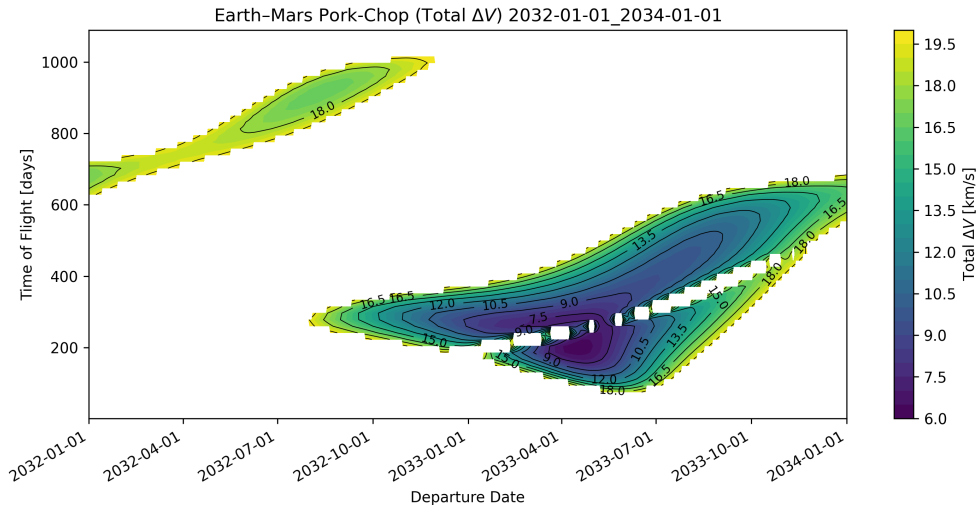
Error Distribution



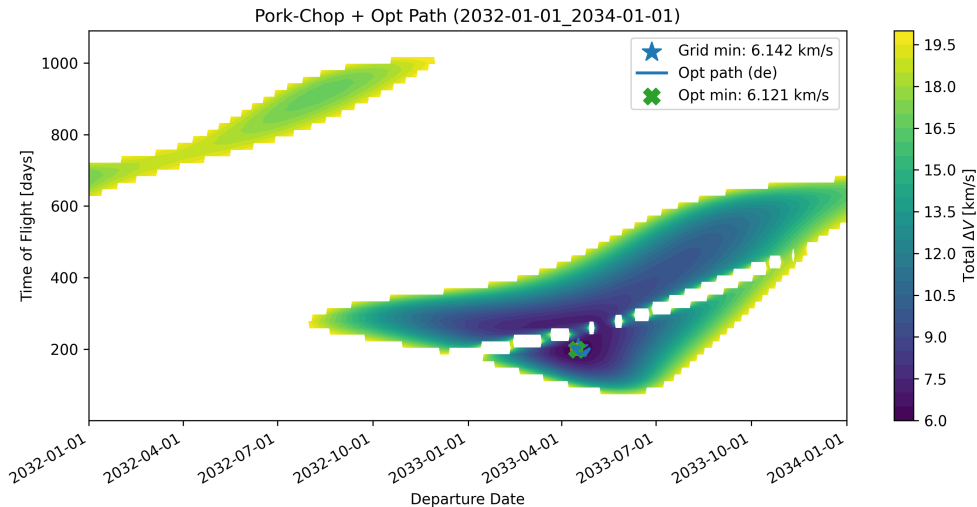
Four-Minima Comparison



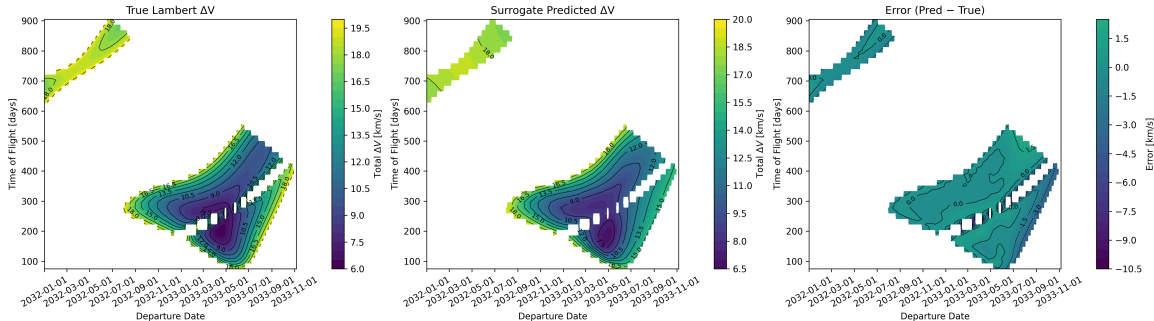
Backup: 2032–2034 Lambert Pork-Chop



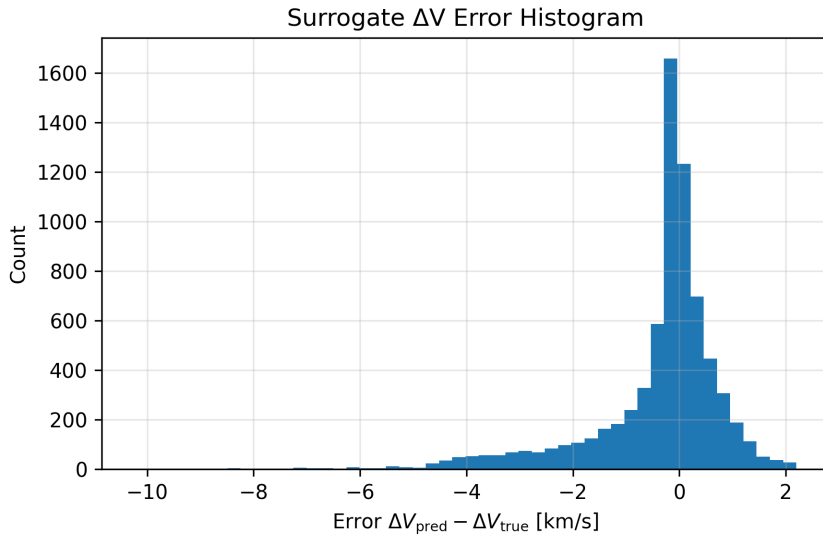
Backup: 2032–2034 True Objective Optimization



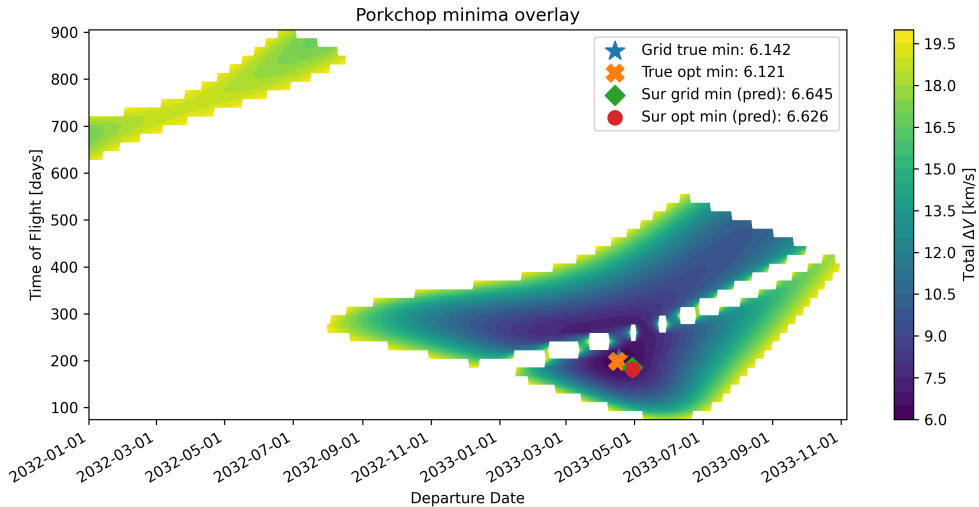
Backup: 2032–2034 Surrogate Triptych



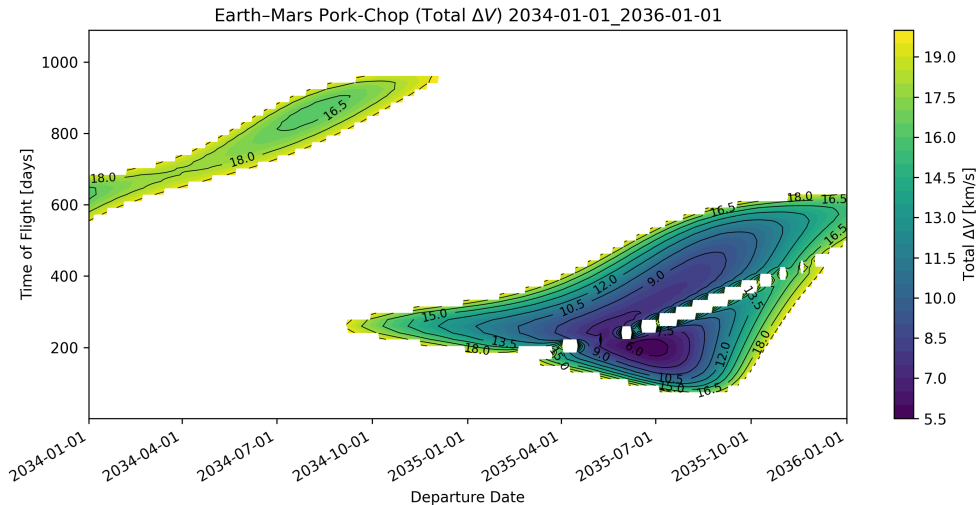
Backup: 2032–2034 Error Histogram



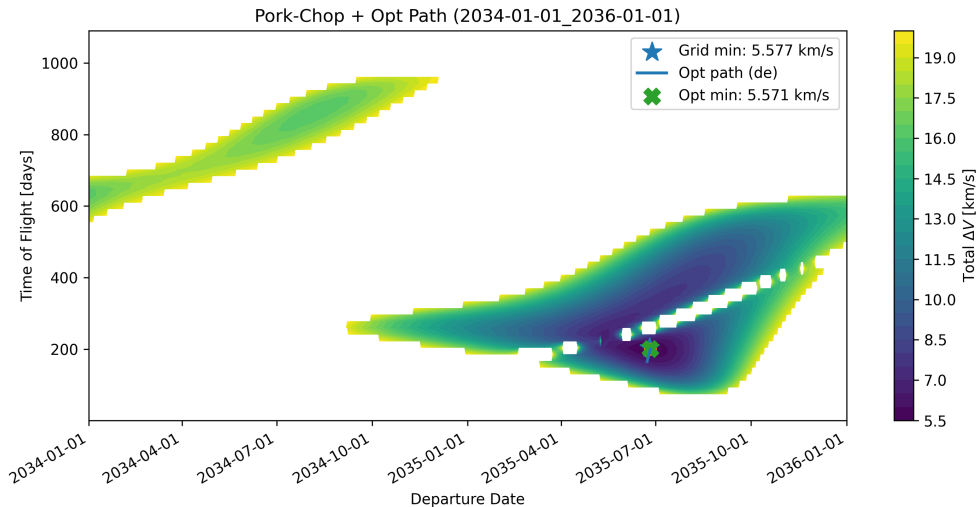
Backup: 2032–2034 Four-Minima Comparison



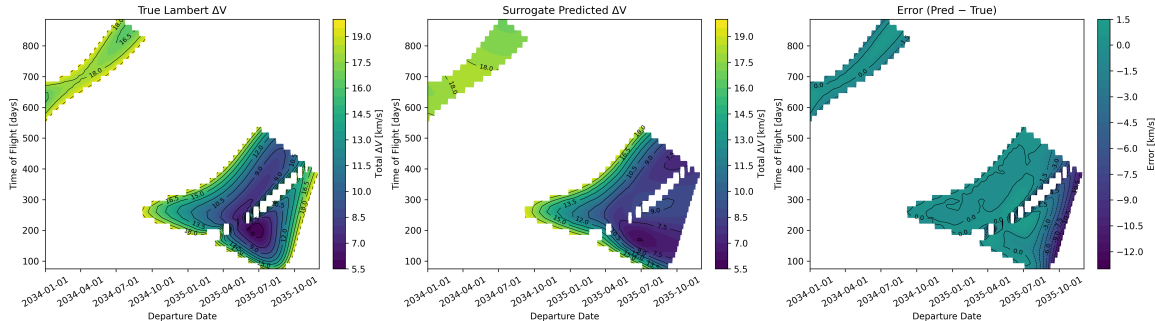
Backup: 2034–2036 Lambert Pork-Chop



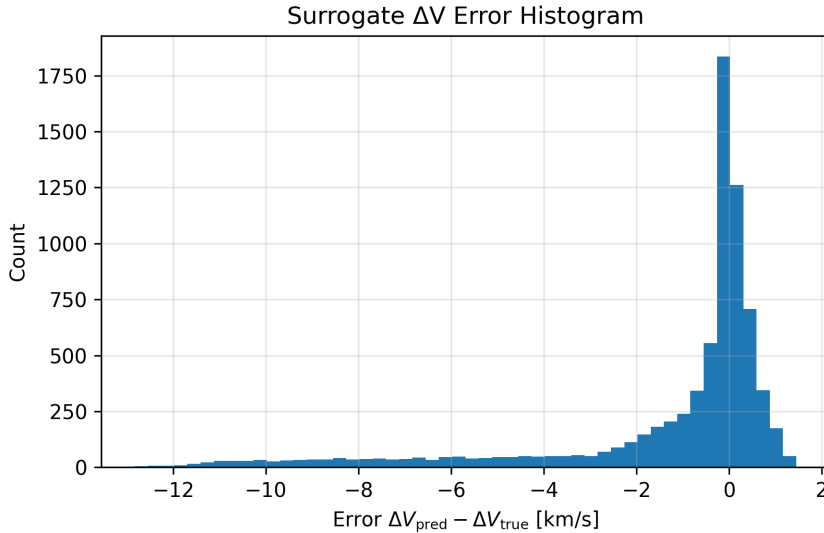
Backup: 2034–2036 True Objective Optimization



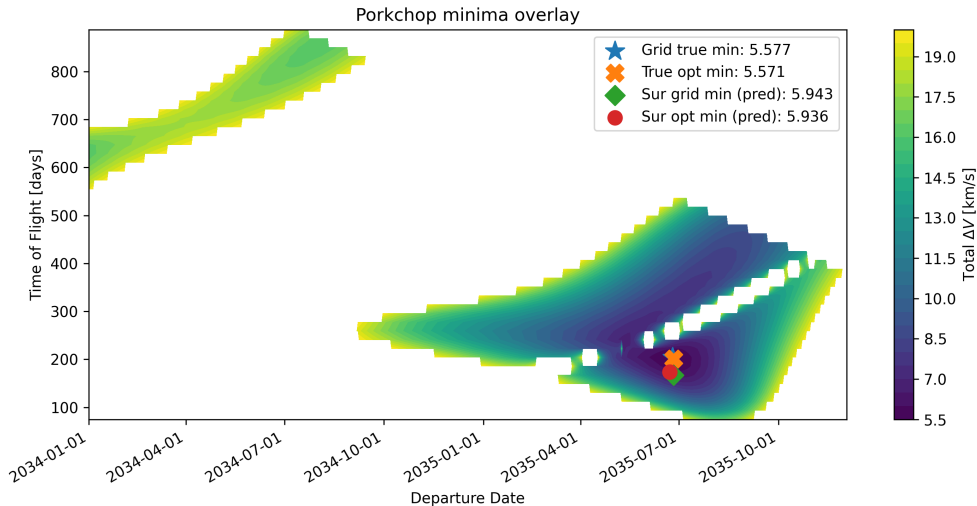
Backup: 2034–2036 Surrogate Triptych



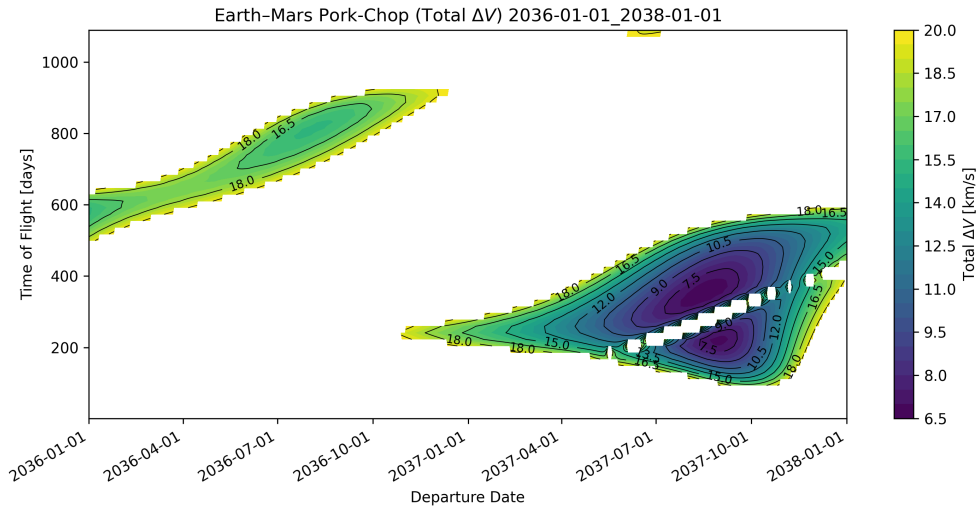
Backup: 2034–2036 Error Histogram



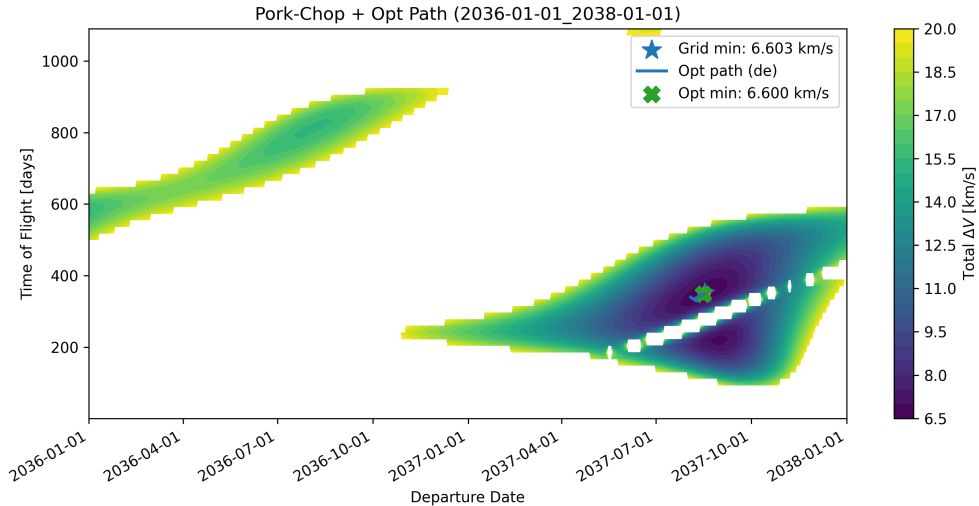
Backup: 2034–2036 Four-Minima Comparison



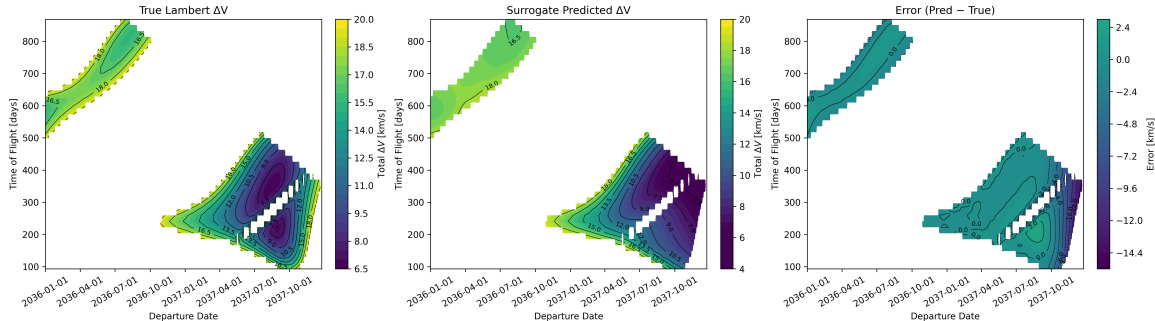
Backup: 2036–2038 Lambert Pork-Chop



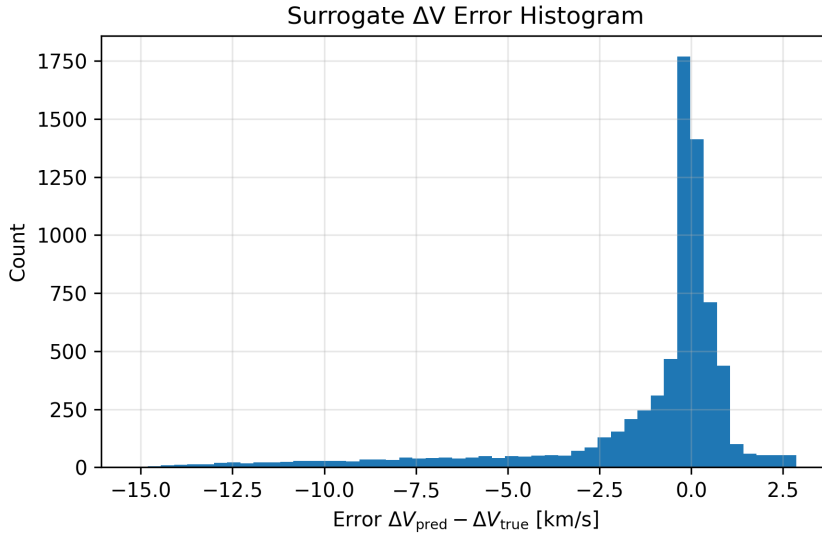
Backup: 2036–2038 True Objective Optimization



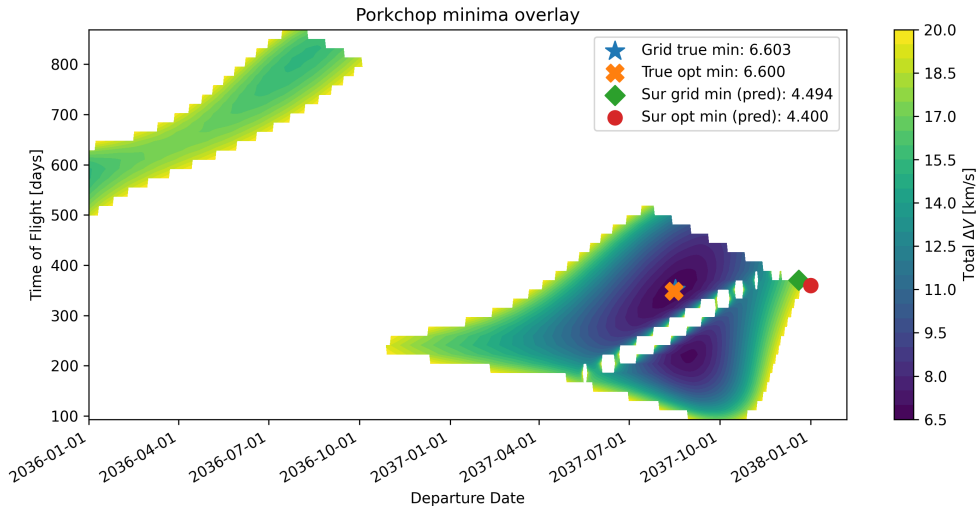
Backup: 2036–2038 Surrogate Triptych



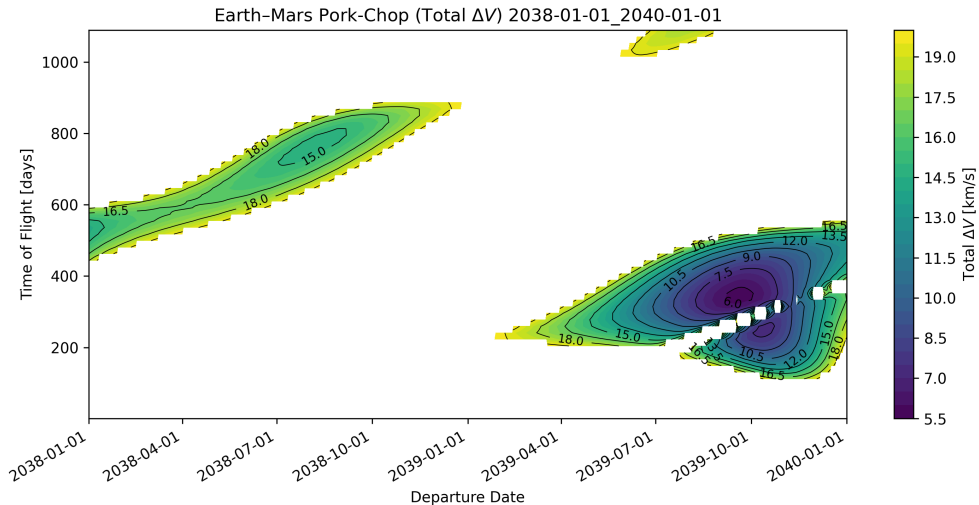
Backup: 2036–2038 Error Histogram



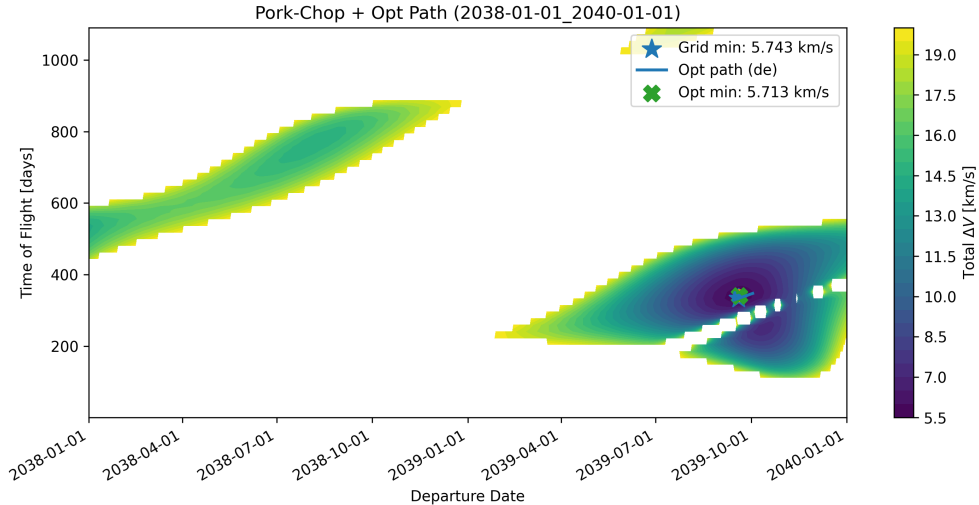
Backup: 2036–2038 Four-Minima Comparison



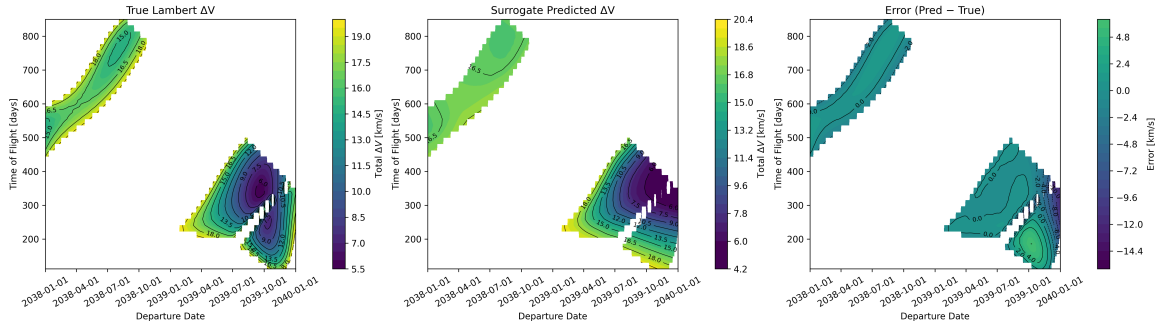
Backup: 2038–2040 Lambert Pork-Chop



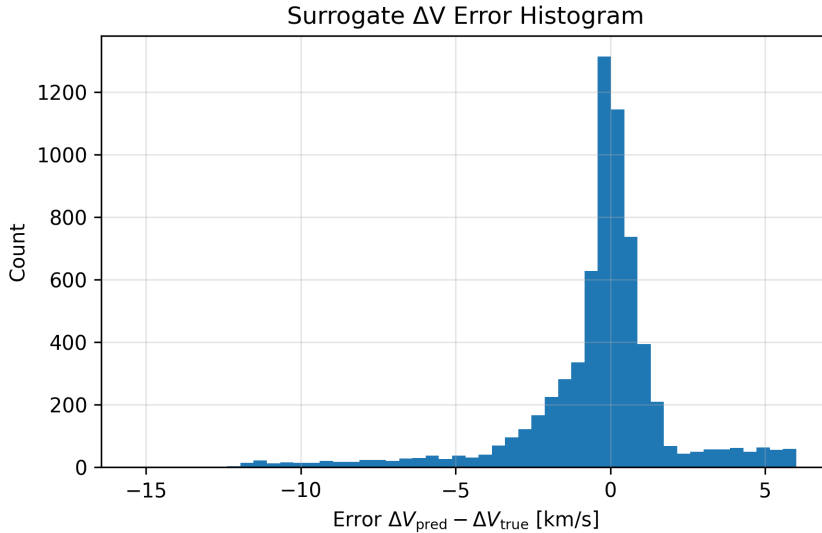
Backup: 2038–2040 True Objective Optimization



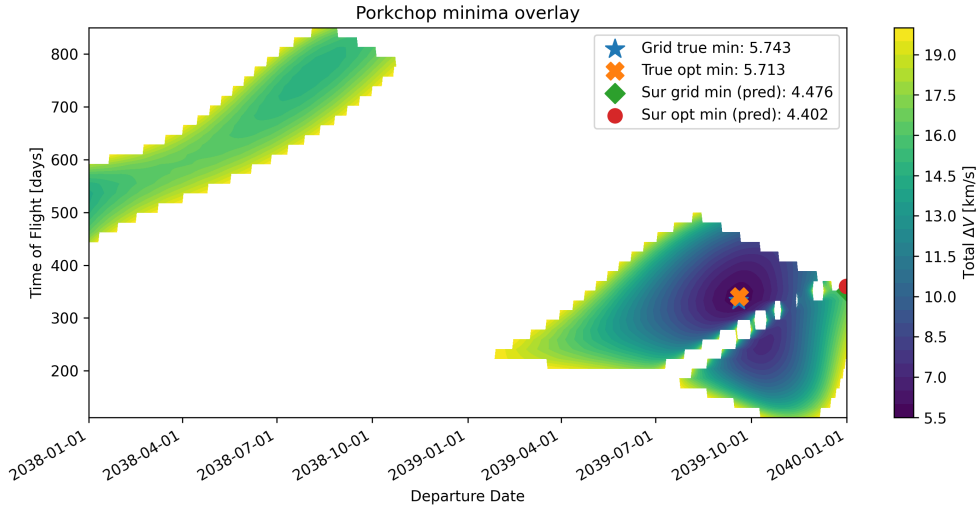
Backup: 2038–2040 Surrogate Triptych



Backup: 2038–2040 Error Histogram



Backup: 2038–2040 Four-Minima Comparison



References



A. Zavoli and L. Federici, "Reinforcement Learning for Low-Thrust Trajectory Design of Interplanetary Missions," AAS 20-616, preprint, 2020.



E. Ruff, R. Russell, M. Stoeckle, P. Miotto, and J. P. How, "Surrogate Neural Networks for Efficient Simulation-based Trajectory Planning Optimization," arXiv preprint arXiv:2303.17468, 2023.



L. Federici and A. Zavoli, "Robust Interplanetary Trajectory Design under Multiple Uncertainties via Meta-Reinforcement Learning," *Acta Astronautica*, 2024.

Questions?

Data-Driven Approximation of Earth–Mars Pork-Chop Surfaces Using Neural-Network Surrogates

Scott Nguyen

*Dept. of Electrical and Computer Engineering
University of New Mexico
Albuquerque, NM, USA*

Abstract—Interplanetary mission design requires repeated evaluation of transfer costs across large launch windows, typically performed by solving Lambert’s problem at thousands of grid points to construct pork-chop plots. While accurate, brute-force Lambert sweeps are computationally expensive and limit rapid trade-space exploration or repeated optimization. This paper presents a data-driven surrogate modeling approach to approximate Earth–Mars pork-chop surfaces by mapping departure epoch and time-of-flight to total ΔV . For each of five two-year windows spanning 2030–2040, Lambert solutions are generated on a dense grid and filtered by a feasibility threshold of $\Delta V \leq 20$ km/s. A compact multi-layer perceptron (MLP) is trained on valid samples only and evaluated using MAE, RMSE, and R^2 on held-out data. In early windows, the surrogate achieves sub-1 km/s MAE and reproduces the dominant low-energy corridors of the Lambert surface. Later windows exhibit larger generalization error and can produce spurious surrogate optima, emphasizing the need for feasibility-aware constraints and validation against the true objective.

Index Terms—interplanetary trajectory design, pork-chop plots, surrogate modeling, Lambert problem, Earth–Mars transfer, neural networks

I. INTRODUCTION

Interplanetary trajectory design explores trade spaces defined by departure epoch, time-of-flight (TOF), and total velocity increment ΔV . Pork-chop plots visualize $\Delta V(t_{\text{dep}}, \text{TOF})$ and reveal low-energy corridors for candidate launch opportunities. Constructing these surfaces via Lambert sweeps requires solving Lambert’s problem at every grid point, which becomes expensive for wide windows or repeated optimization.

Machine learning can reduce repeated computation via surrogate models that approximate expensive physics-based objectives. Surrogate neural networks have accelerated simulation-based optimization loops by replacing repeated forward calls with fast regressors [2]. Reinforcement learning and meta-RL have also been explored for robust trajectory design under uncertainty [1], [3]. This work focuses on supervised learning for impulsive Earth–Mars Lambert pork-chop approximation.

A. Contributions

- A reproducible pipeline for generating Earth–Mars Lambert datasets across five two-year windows from 2030 to 2040.

- A compact MLP surrogate for ΔV prediction from $(t_{\text{dep}}, \text{TOF})$.
- Quantitative and qualitative assessment of surrogate fidelity and optimization behavior across windows.

II. BACKGROUND

Lambert’s problem provides an impulsive two-body approximation for connecting heliocentric endpoints in a fixed time. Sweeping departure date and TOF yields pork-chop surfaces whose valleys correspond to favorable geometries. Data-driven surrogates aim to approximate these surfaces with orders-of-magnitude lower evaluation cost. Prior work demonstrated surrogate networks in simulation-based optimization [2] and learning-based robust trajectory methods [1], [3]. The present study isolates the supervised surrogate approximation problem for impulsive Earth–Mars transfers.

III. OBJECTIVE AND PROBLEM FORMULATION

A. Two-Body Transfer Model

We assume heliocentric two-body dynamics during the transfer arc:

$$\ddot{\mathbf{r}}(t) = -\mu_{\odot} \frac{\mathbf{r}(t)}{\|\mathbf{r}(t)\|^3}. \quad (1)$$

Decision variables are the departure epoch and TOF:

$$x \triangleq \begin{bmatrix} t_{\text{dep}} \\ \text{TOF} \end{bmatrix}, \quad t_{\text{arr}} = t_{\text{dep}} + \text{TOF}. \quad (2)$$

Planetary ephemerides provide Earth and Mars heliocentric states $(\mathbf{r}_E(t), \mathbf{v}_E(t))$ and $(\mathbf{r}_M(t), \mathbf{v}_M(t))$.

B. Lambert Boundary-Value Map

For a given $(t_{\text{dep}}, \text{TOF})$, Lambert’s problem solves for the transfer velocities that connect

$$\mathbf{r}(t_{\text{dep}}) = \mathbf{r}_E(t_{\text{dep}}), \quad \mathbf{r}(t_{\text{arr}}) = \mathbf{r}_M(t_{\text{arr}}) \quad (3)$$

under (1), returning

$$(\mathbf{v}_{\text{dep}}^*, \mathbf{v}_{\text{arr}}^*) = \text{Lambert}(\mathbf{r}_E(t_{\text{dep}}), \mathbf{r}_M(t_{\text{arr}}), \text{TOF}; \mu_{\odot}). \quad (4)$$

C. Impulsive ΔV Objective Derivation

In an impulsive approximation, the spacecraft instantaneously changes velocity to enter and exit the transfer arc. The departure burn is the mismatch between the required transfer departure velocity and Earth's heliocentric velocity:

$$\Delta V_{\text{dep}}(t_{\text{dep}}, \text{TOF}) = \|\mathbf{v}_{\text{dep}}^* - \mathbf{v}_E(t_{\text{dep}})\|. \quad (5)$$

The arrival burn is the mismatch between Mars' heliocentric velocity and the transfer arrival velocity:

$$\Delta V_{\text{arr}}(t_{\text{dep}}, \text{TOF}) = \|\mathbf{v}_M(t_{\text{arr}}) - \mathbf{v}_{\text{arr}}^*\|. \quad (6)$$

The total two-impulse cost is the sum:

$$J(t_{\text{dep}}, \text{TOF}) \triangleq \Delta V_{\text{tot}} = \Delta V_{\text{dep}}(t_{\text{dep}}, \text{TOF}) + \Delta V_{\text{arr}}(t_{\text{dep}}, \text{TOF}). \quad (7)$$

The pork-chop surface is the evaluation of (7) over a grid in $(t_{\text{dep}}, \text{TOF})$.

D. Feasibility Filtering

To restrict learning to relevant and numerically stable regions, solutions with

$$J(t_{\text{dep}}, \text{TOF}) > \Delta V_{\text{max}}, \quad \Delta V_{\text{max}} = 20 \text{ km/s} \quad (8)$$

are discarded (treated as invalid).

IV. METHODOLOGY

A. Dataset Generation

For each two-year window, departure epochs are sampled daily (≈ 731 – 732 departures). For each departure, $N_{\text{tof}} = 60$ TOF values are sampled uniformly within bounds consistent with the arrival window, producing a grid of $\sim 44,000$ Lambert solves per window. At each grid point, the pipeline:

- 1) computes t_{arr} via (2);
- 2) queries ephemerides for Earth/Mars heliocentric states;
- 3) solves Lambert via (4);
- 4) computes J via (7);
- 5) applies feasibility filter (8).

After filtering, each window retains roughly 6,700–7,500 valid samples for learning.

B. Surrogate Model and Training

A compact MLP is trained per-window to approximate

$$\hat{J}(t_{\text{dep}}, \text{TOF}) \approx J(t_{\text{dep}}, \text{TOF}) \quad (9)$$

using only valid samples. Inputs are normalized using per-window mean/std statistics. Data are split 70%/15%/15% into train/validation/test. Training uses early stopping based on validation loss.

C. Optimization and Evaluation

To assess the surrogate beyond pointwise regression, differential evolution (DE) minimizes both the true objective J (Lambert) and the surrogate objective $\hat{J}$ over continuous bounds. The surrogate-predicted optimum is then evaluated using the true Lambert model to determine whether the surrogate's minimum corresponds to a physically meaningful low- ΔV transfer.

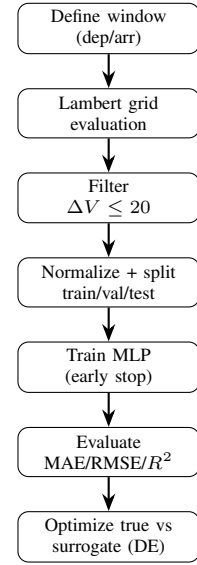


Fig. 1. End-to-end workflow for dataset generation, surrogate training, and optimization comparison.

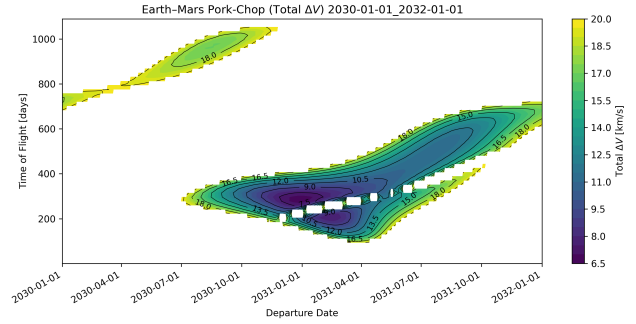


Fig. 2. Lambert pork-chop surface for 2030–2032.

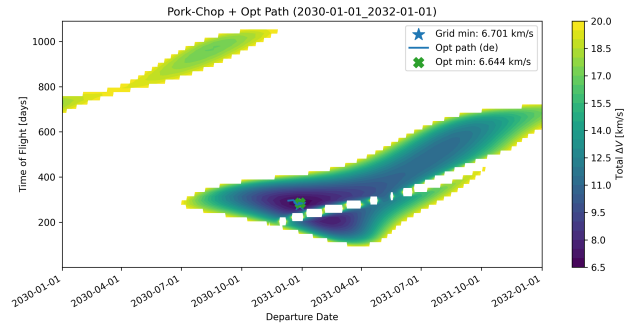


Fig. 3. True-objective optimization path (DE) for 2030–2032.

D. Workflow Flowchart

V. RESULTS

A. Baseline Surface and Optimization Path

Fig. 2 shows the Lambert pork-chop surface for the 2030–2032 window. Fig. 3 overlays the DE optimization path for the true Lambert objective.

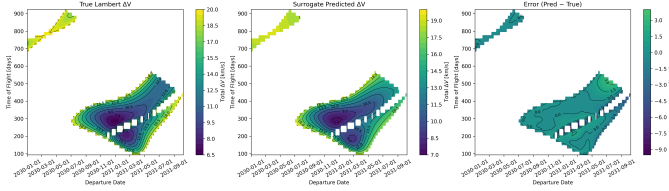


Fig. 4. Triptych for 2030–2032: true surface, surrogate prediction, and error.

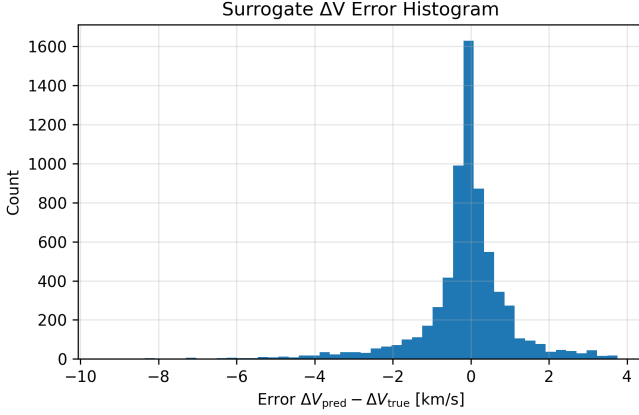


Fig. 5. Error histogram for 2030–2032.

TABLE I
SUMMARY OF MINIMA AND SURROGATE ERROR METRICS ACROSS WINDOWS

Window	Grid min [km/s]	True opt min [km/s]	MAE [km/s]	RMSE [km/s]
2030–2032	6.7009	6.6445	0.7313	1.2269
2032–2034	6.1420	6.1211	0.7985	1.3743
2034–2036	5.5770	5.5707	1.5362	3.0171
2036–2038	6.6032	6.6002	1.5831	3.1374
2038–2040	5.7429	5.7125	1.4953	2.5518

B. Surrogate Fidelity and Error Distributions

Fig. 4 compares true vs surrogate and error for 2030–2032, while Fig. 5 shows the error histogram.

C. Minima and Aggregate Metrics

Table I summarizes Lambert minima and surrogate errors across all windows.

D. Four-Minima Comparison

Fig. 6 shows the four-minima comparison for 2030–2032.

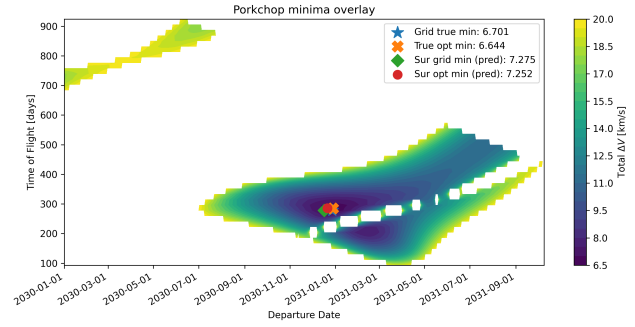


Fig. 6. Four-minima overlay for 2030–2032.

VI. DISCUSSION

The surrogate reproduces dominant low- ΔV corridors for early windows and achieves sub-1 km/s MAE. Later windows show higher error and more failure cases during surrogate-based optimization, consistent with sharper gradients, fewer valid samples, and more irregular feasible regions. Because DE explores continuous bounds, the surrogate can be exploited in regions where training data are sparse; validating J at surrogate optima is therefore essential. Future improvements include feasibility-aware modeling (joint feasibility classification), uncertainty-aware regression, and multi-window training.

VII. CONCLUSION

This work demonstrated a supervised surrogate for approximating Earth–Mars Lambert pork-chop surfaces. Results show meaningful accuracy for early windows and highlight limitations for later windows, especially under continuous optimization. The approach supports fast trade-space exploration for preliminary mission design while motivating feasibility-aware extensions.

REFERENCES

- [1] A. Zavoli and L. Federici, “Reinforcement learning for robust trajectory design of interplanetary missions,” *Journal of Guidance, Control, and Dynamics*, vol. 44, no. 8, pp. 1447–1459, 2021.
- [2] E. Ruff, R. Russell, M. Stoeckle, P. Miotto, and J. P. How, “Surrogate neural networks for efficient simulation-based trajectory planning optimization,” arXiv preprint arXiv:2303.17468, 2023.
- [3] L. Federici and A. Zavoli, “Robust interplanetary trajectory design under multiple uncertainties via meta-reinforcement learning,” *Acta Astronautica*, 2024.

APPENDIX

APPENDIX: ADDITIONAL WINDOWS (BACKUP FIGURES)

2032–2034

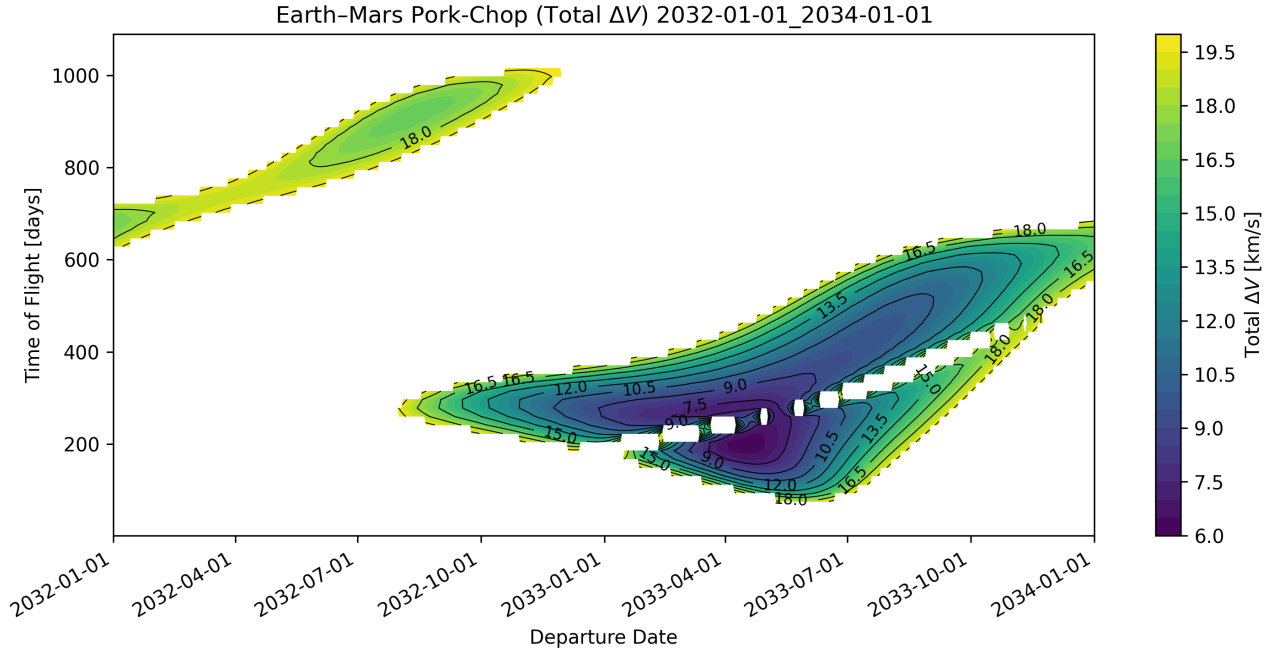


Fig. 7. Lambert pork-chop surface for 2032–2034.

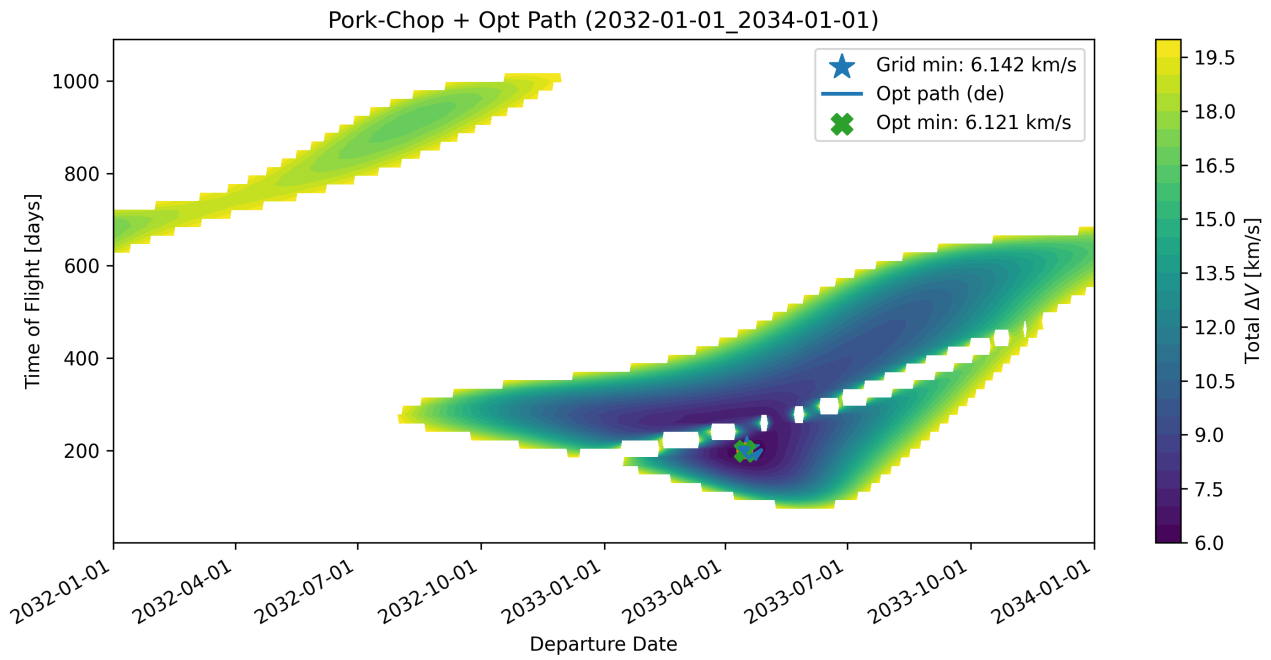


Fig. 8. True-objective optimization path for 2032–2034.

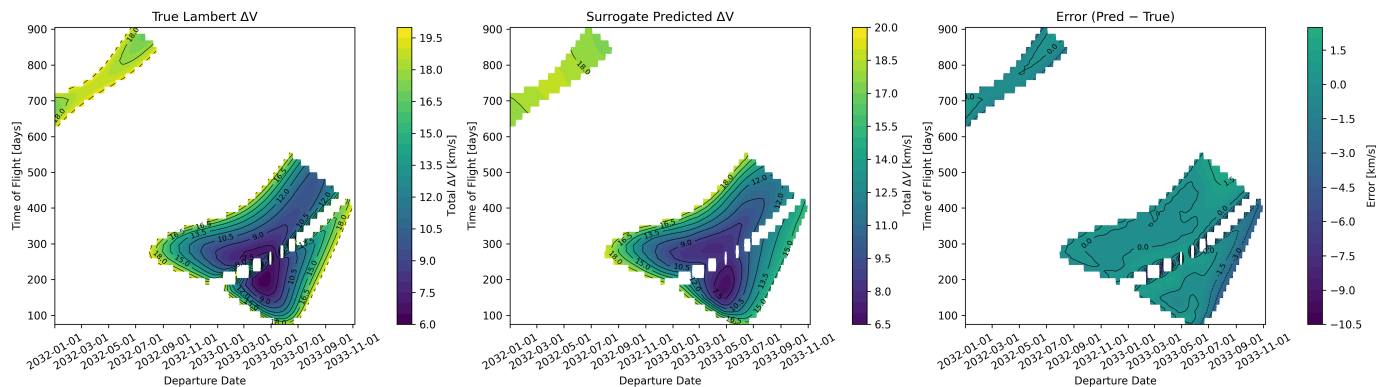


Fig. 9. Triptych for 2032–2034: true surface, surrogate prediction, and error.

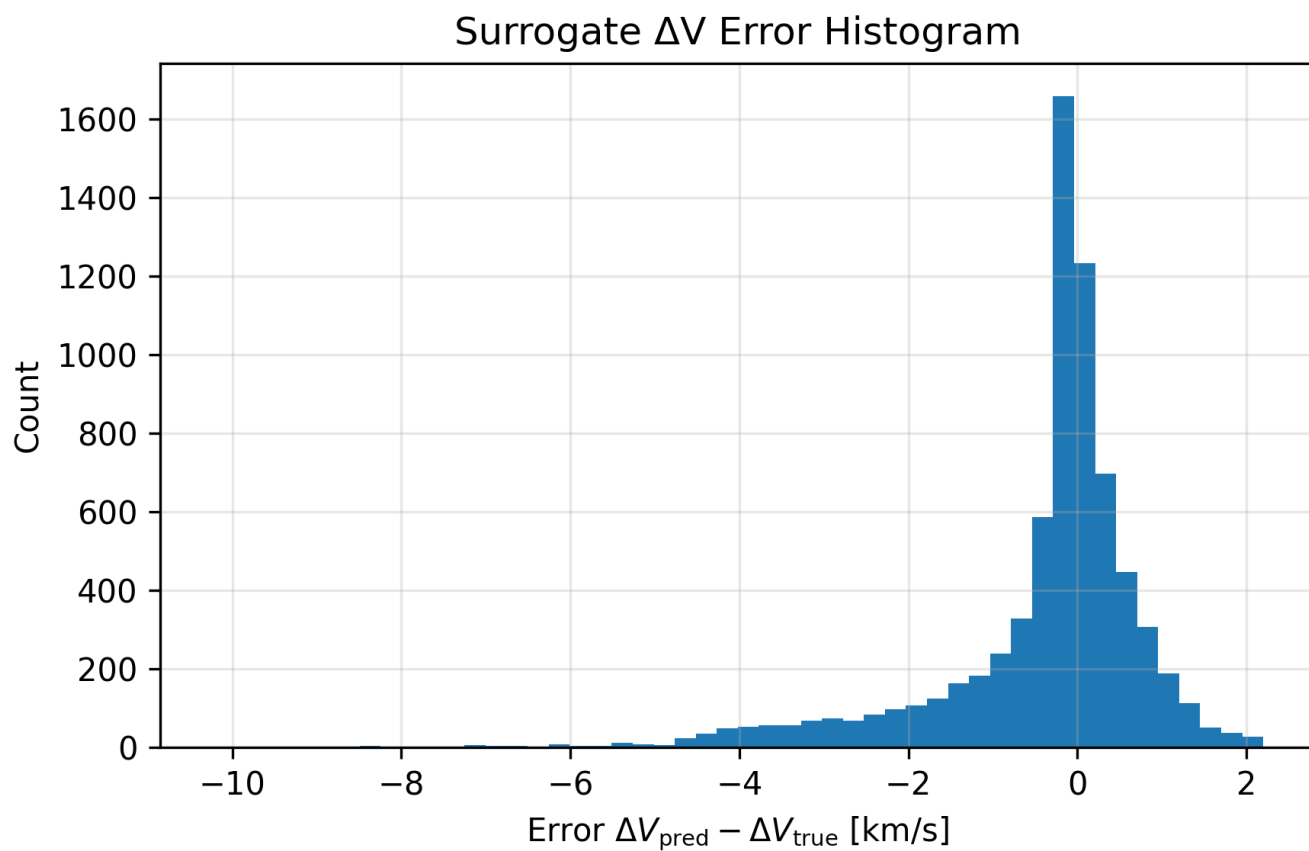


Fig. 10. Error histogram for 2032–2034.

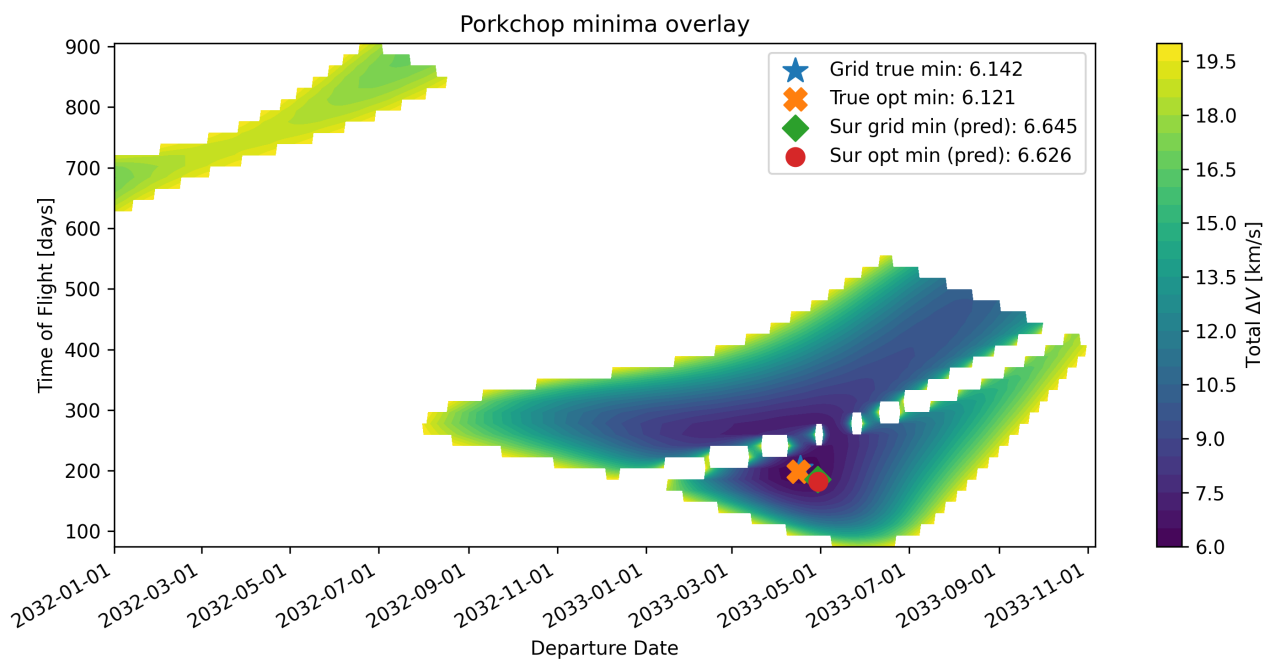


Fig. 11. Four-minima overlay for 2032–2034.

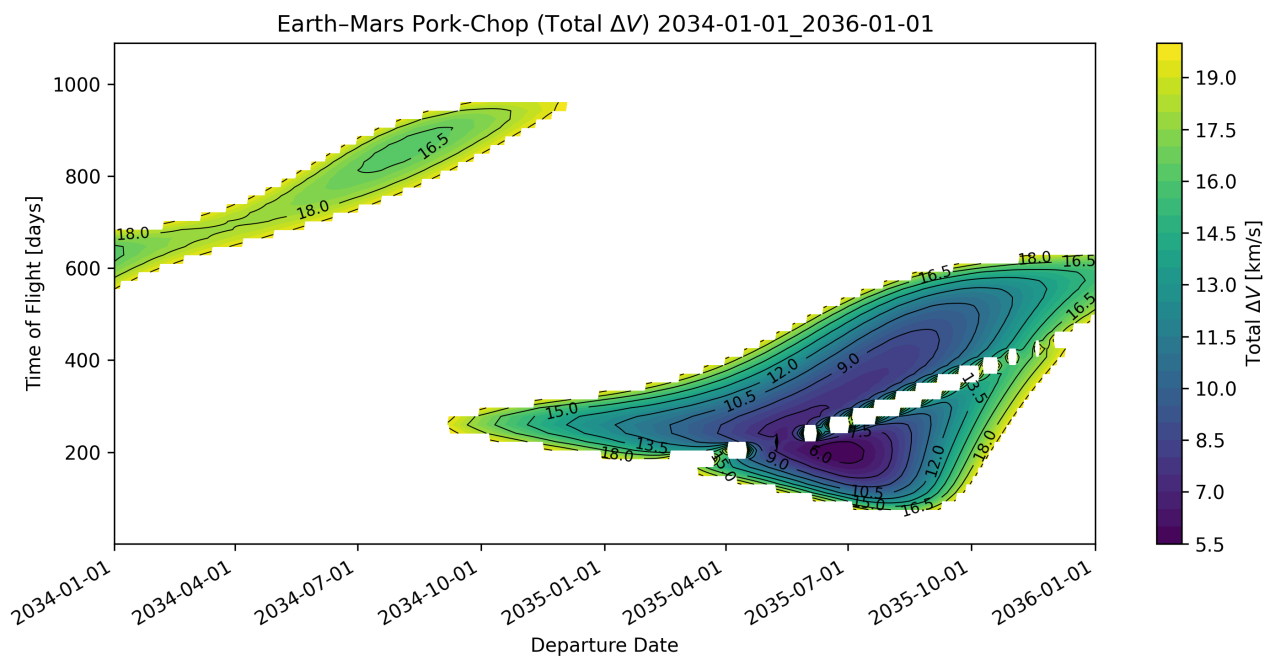


Fig. 12. Lambert pork-chop surface for 2034–2036.

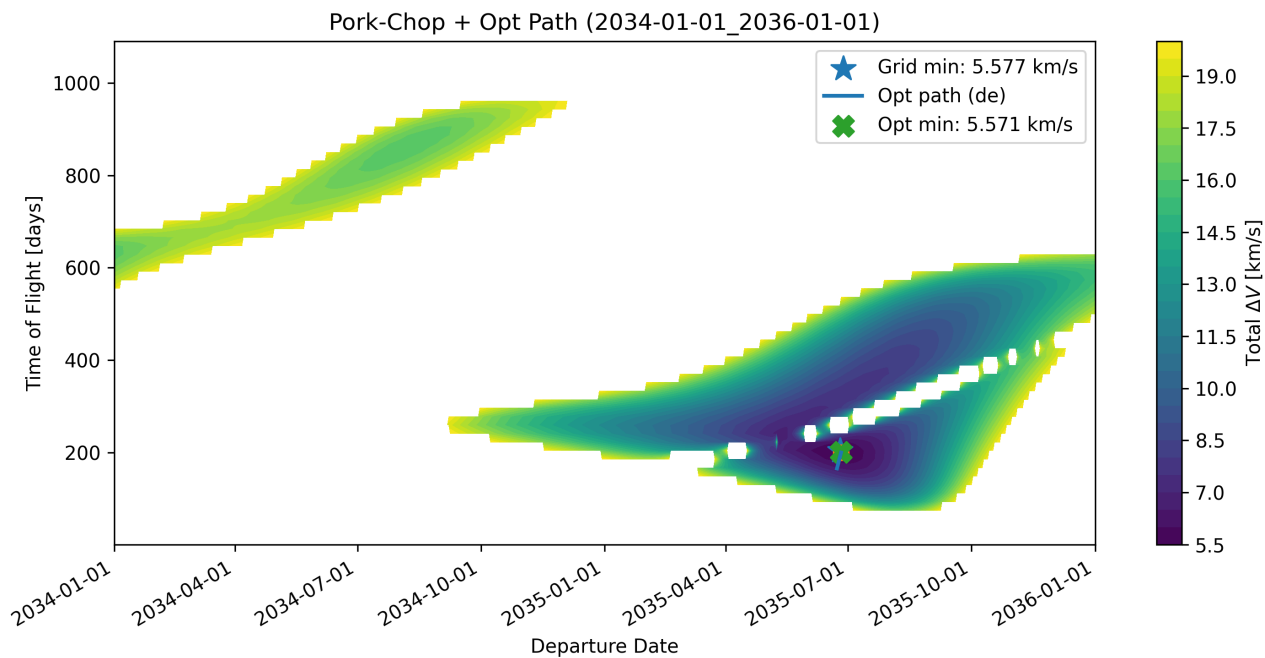


Fig. 13. True-objective optimization path for 2034–2036.

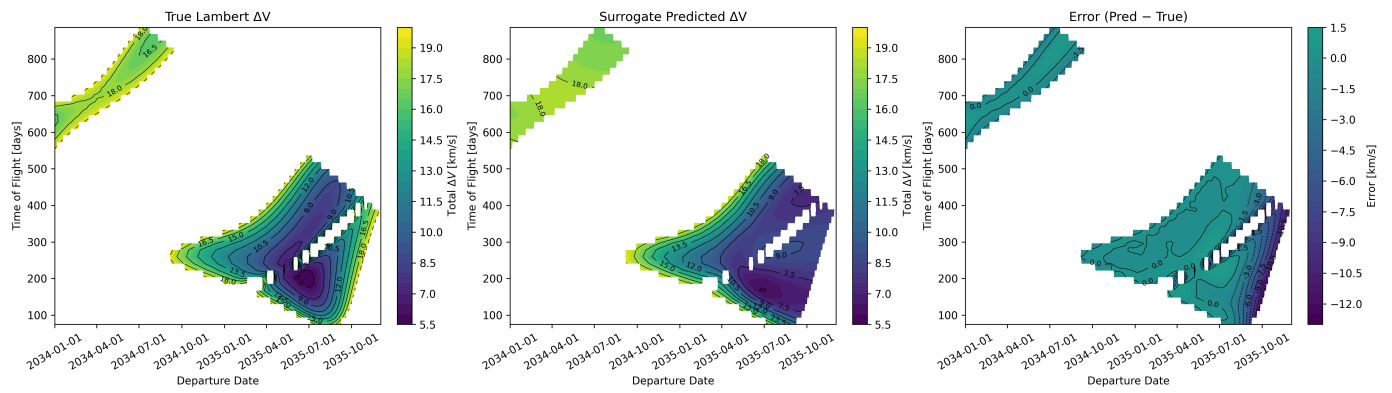


Fig. 14. Triptych for 2034–2036: true surface, surrogate prediction, and error.

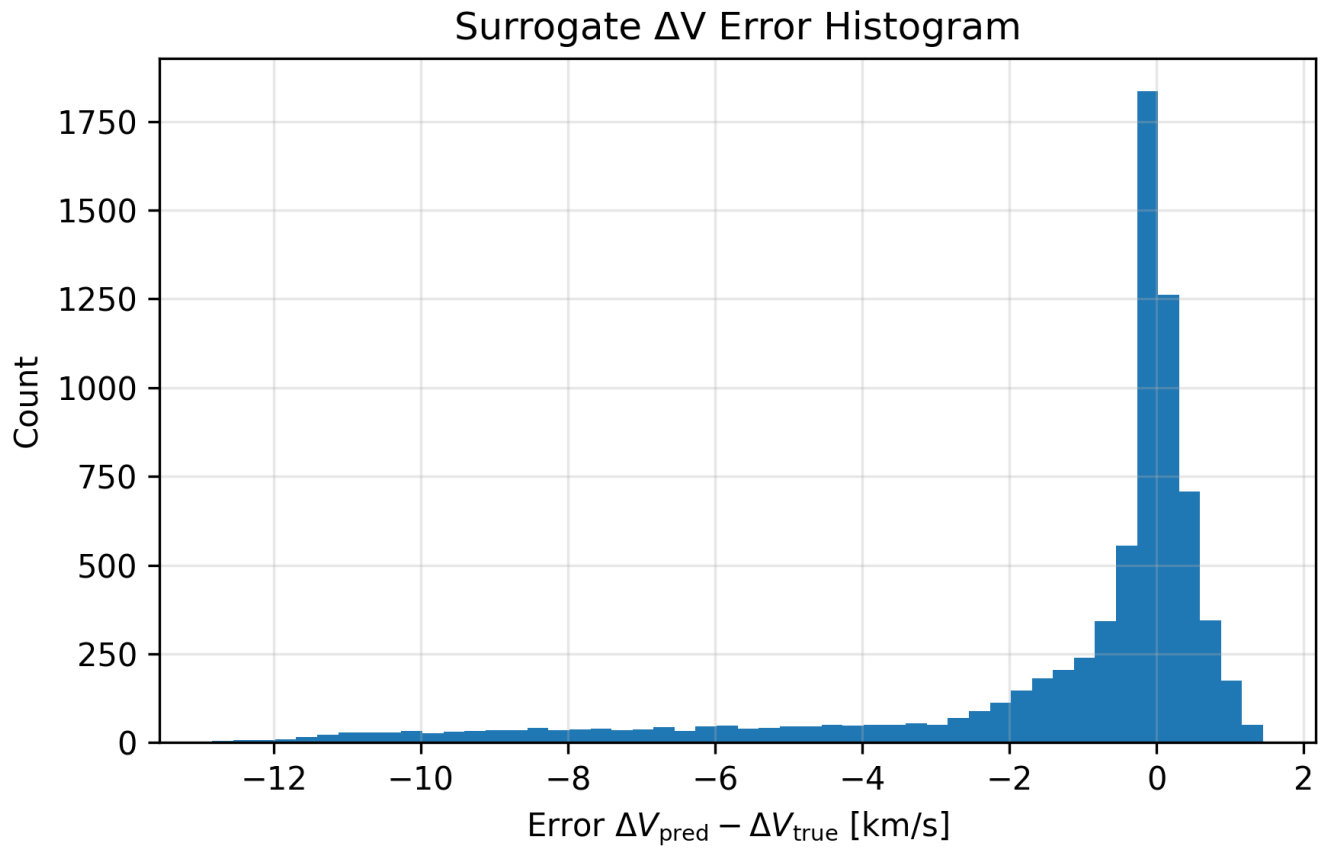


Fig. 15. Error histogram for 2034–2036.

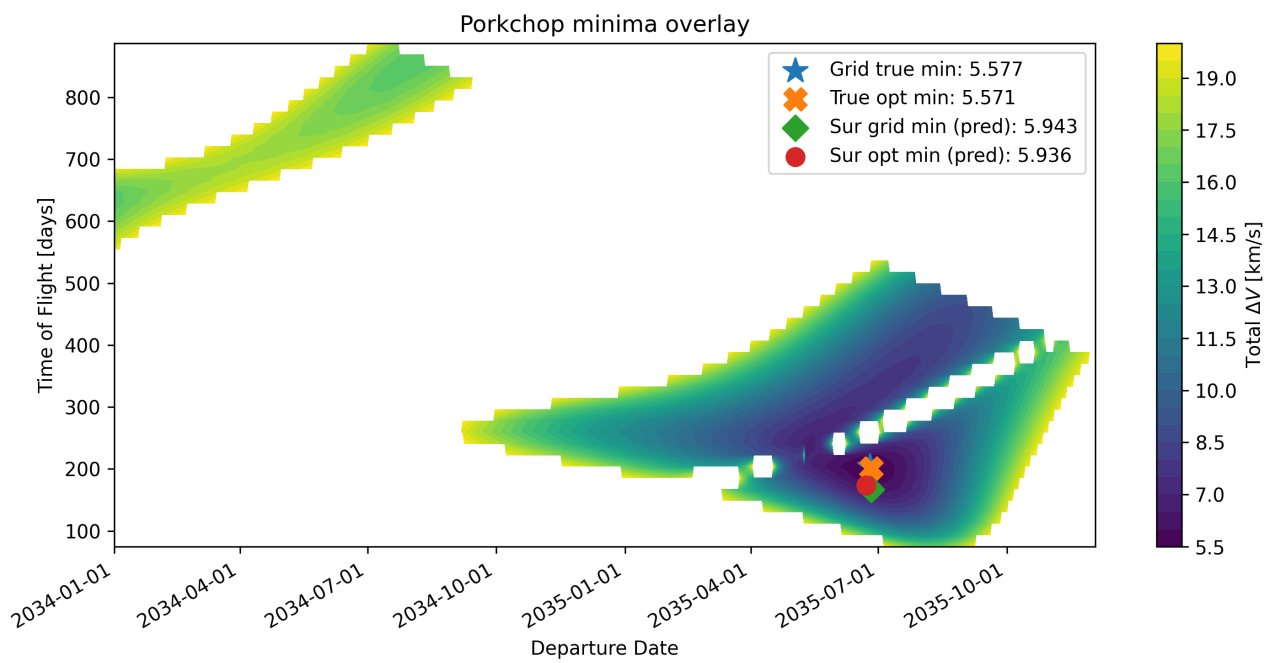


Fig. 16. Four-minima overlay for 2034–2036.

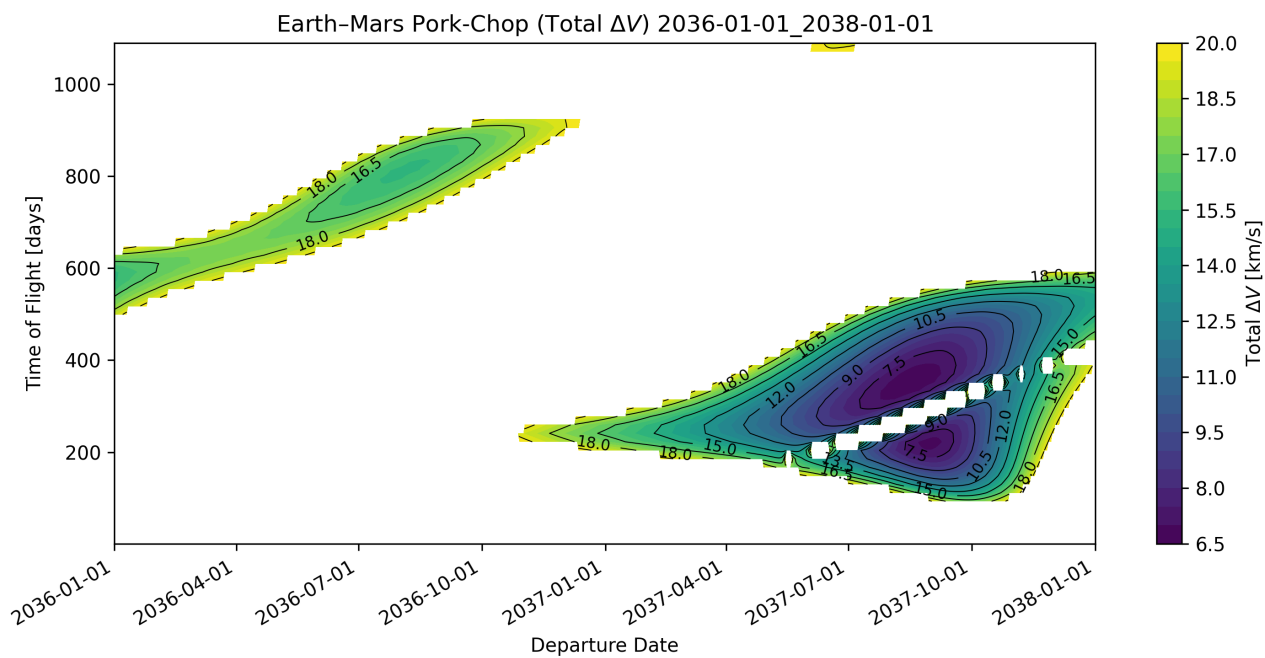


Fig. 17. Lambert pork-chop surface for 2036–2038.

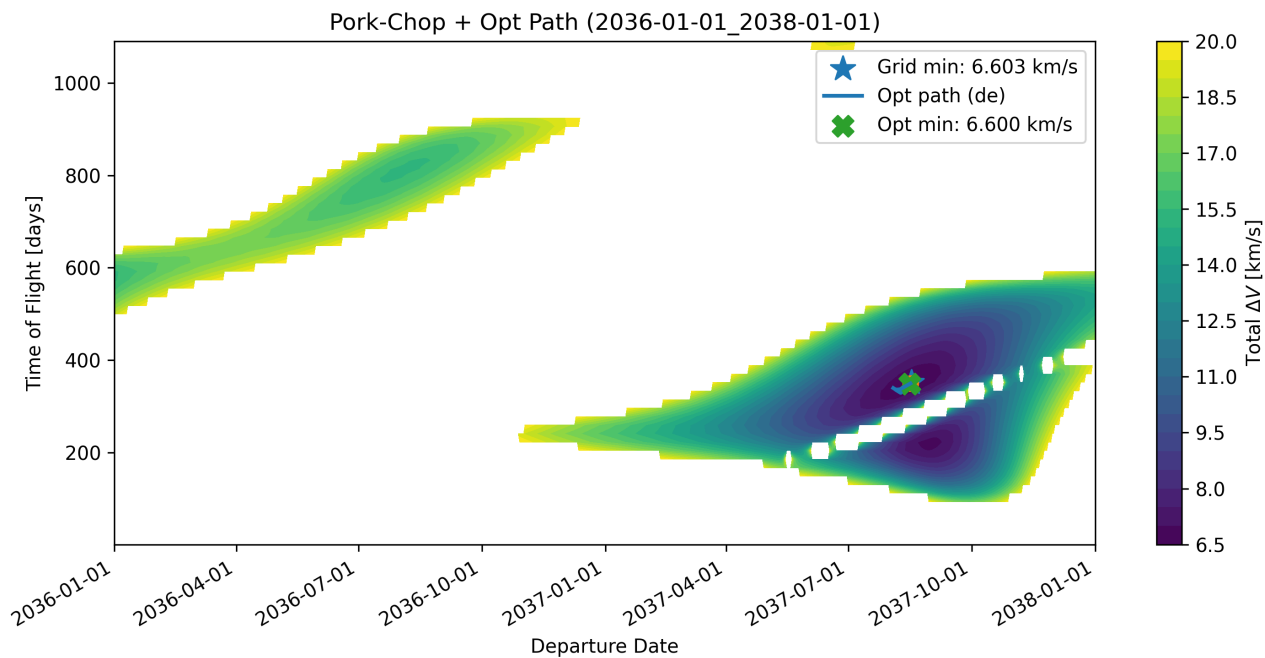


Fig. 18. True-objective optimization path for 2036–2038.

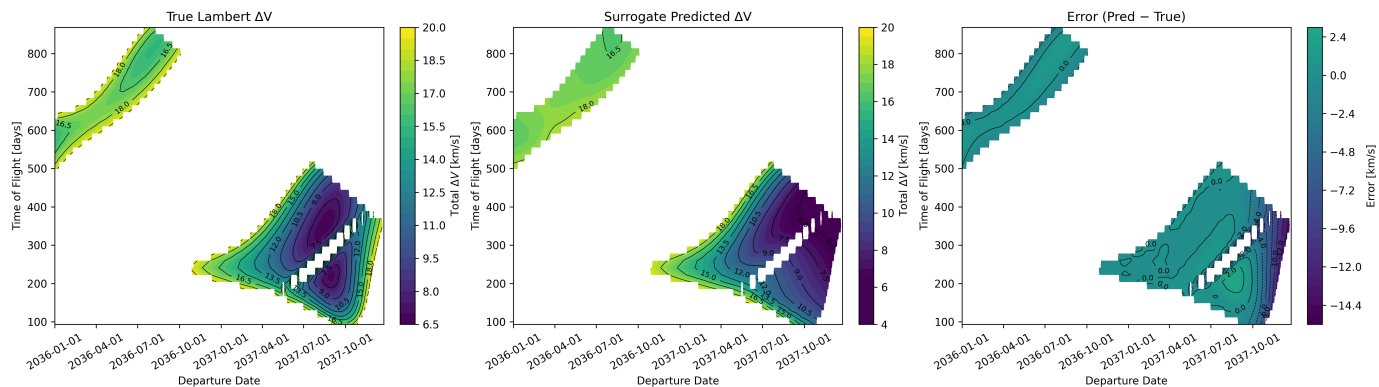


Fig. 19. Triptych for 2036–2038: true surface, surrogate prediction, and error.

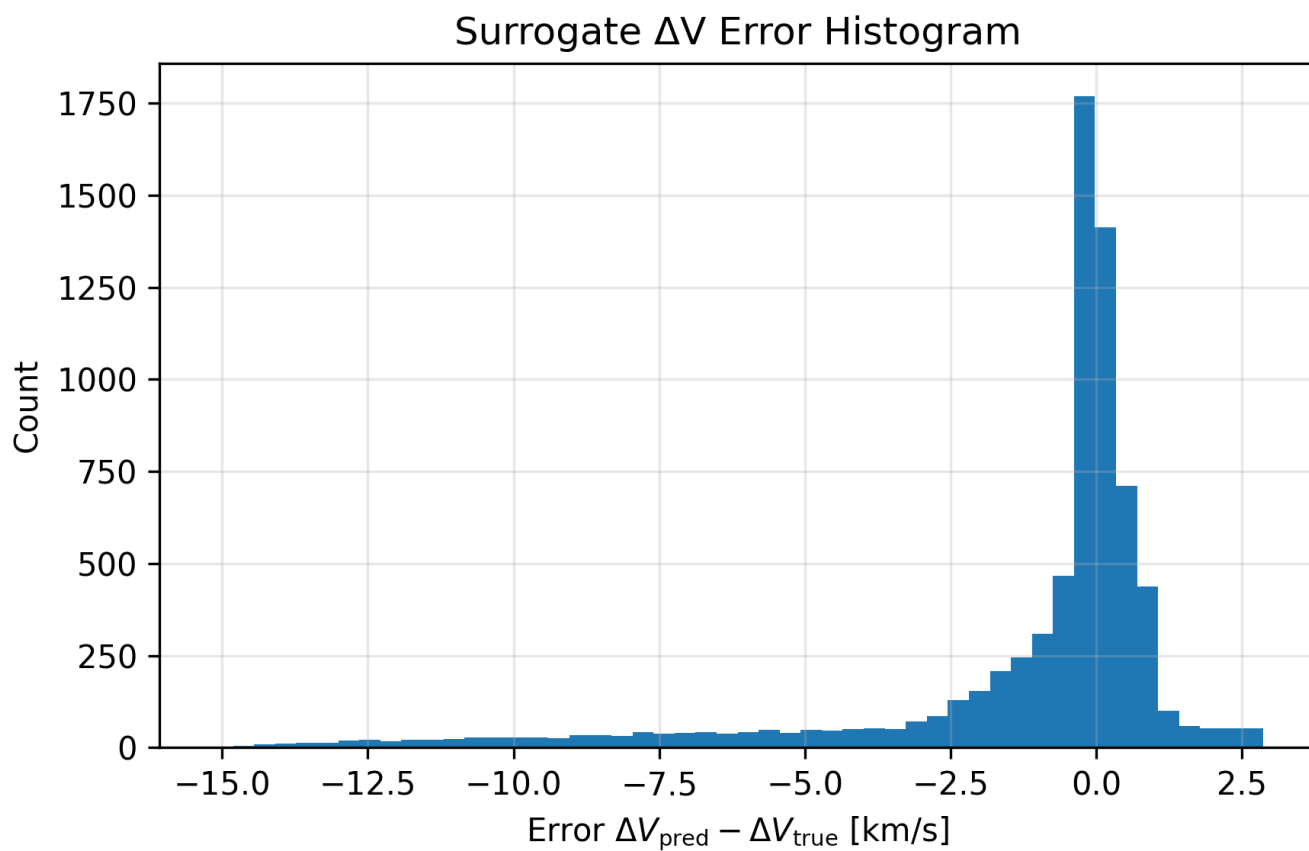


Fig. 20. Error histogram for 2036–2038.

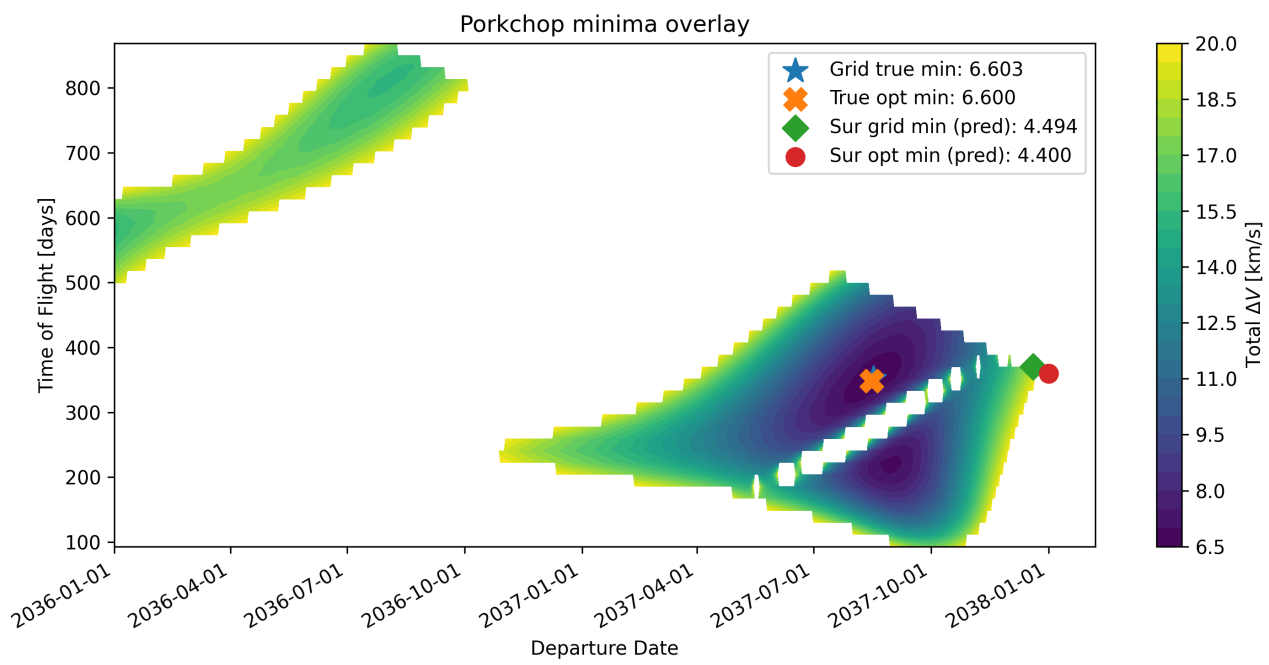


Fig. 21. Four-minima overlay for 2036–2038.

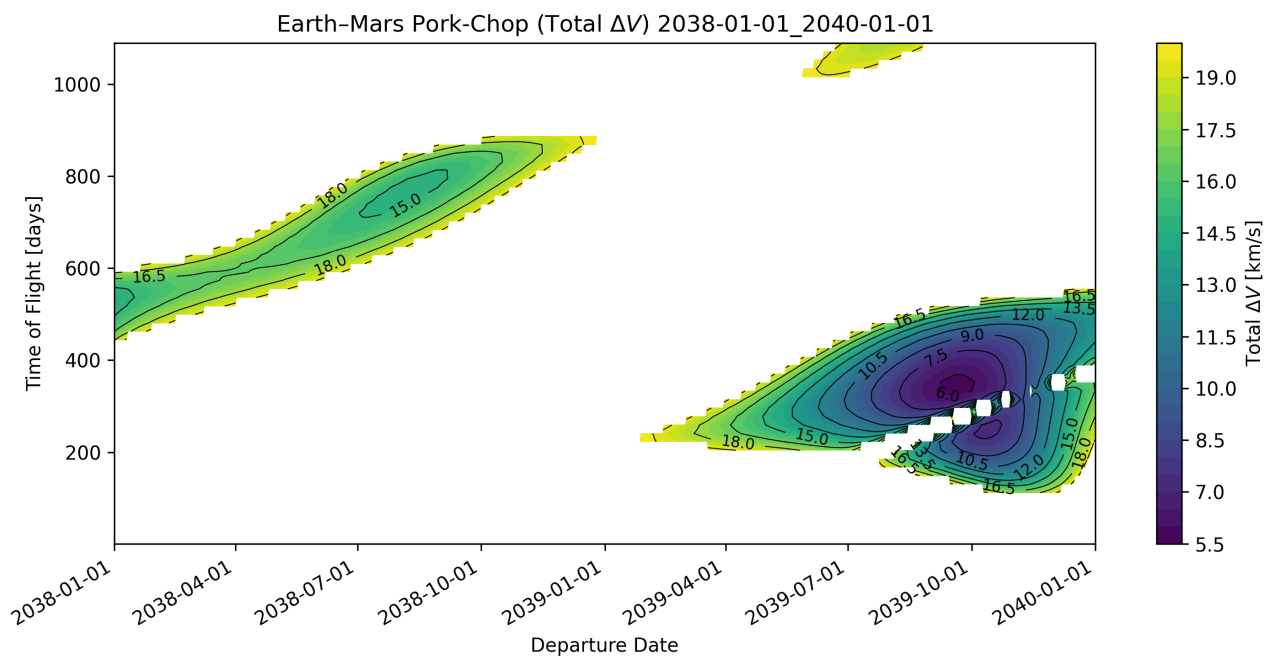


Fig. 22. Lambert pork-chop surface for 2038–2040.

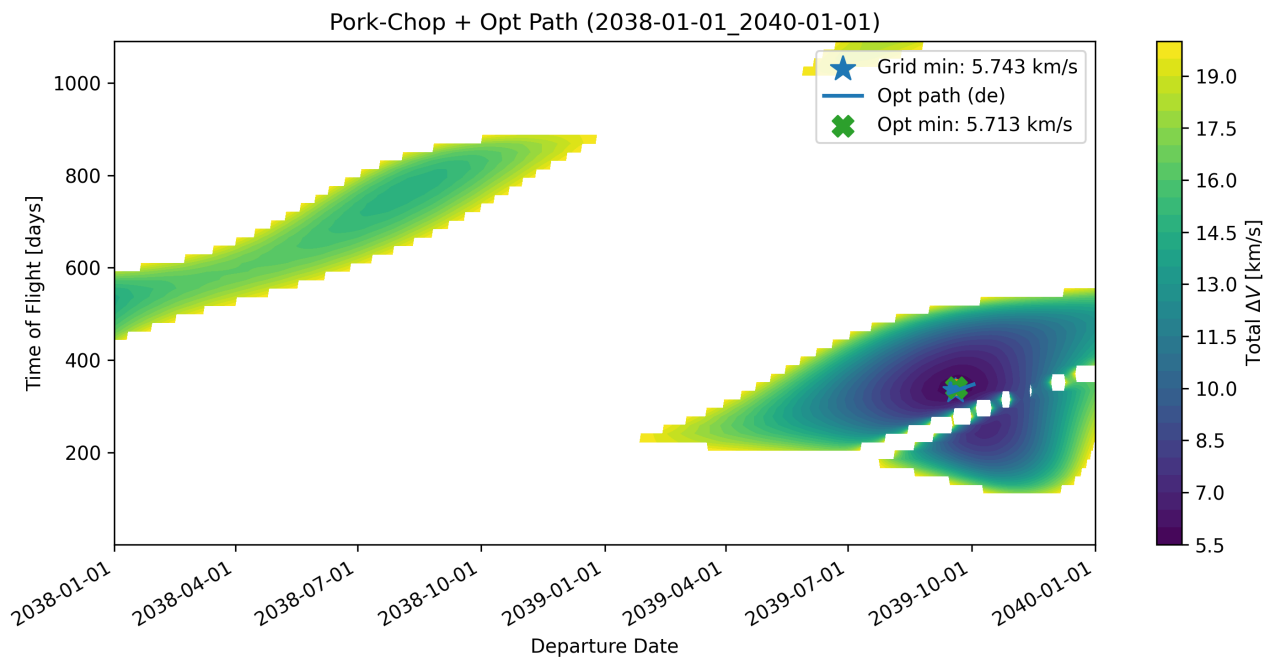


Fig. 23. True-objective optimization path for 2038–2040.

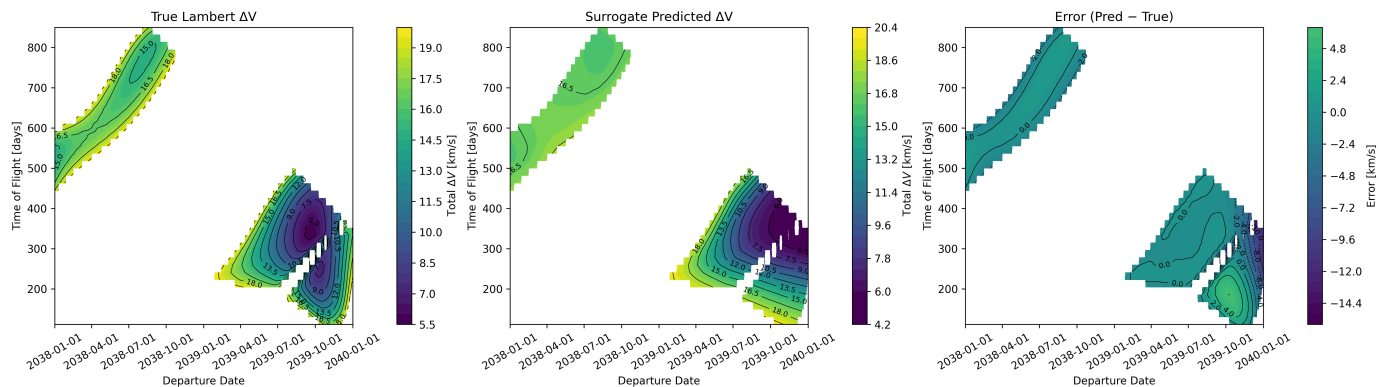


Fig. 24. Triptych for 2038–2040: true surface, surrogate prediction, and error.

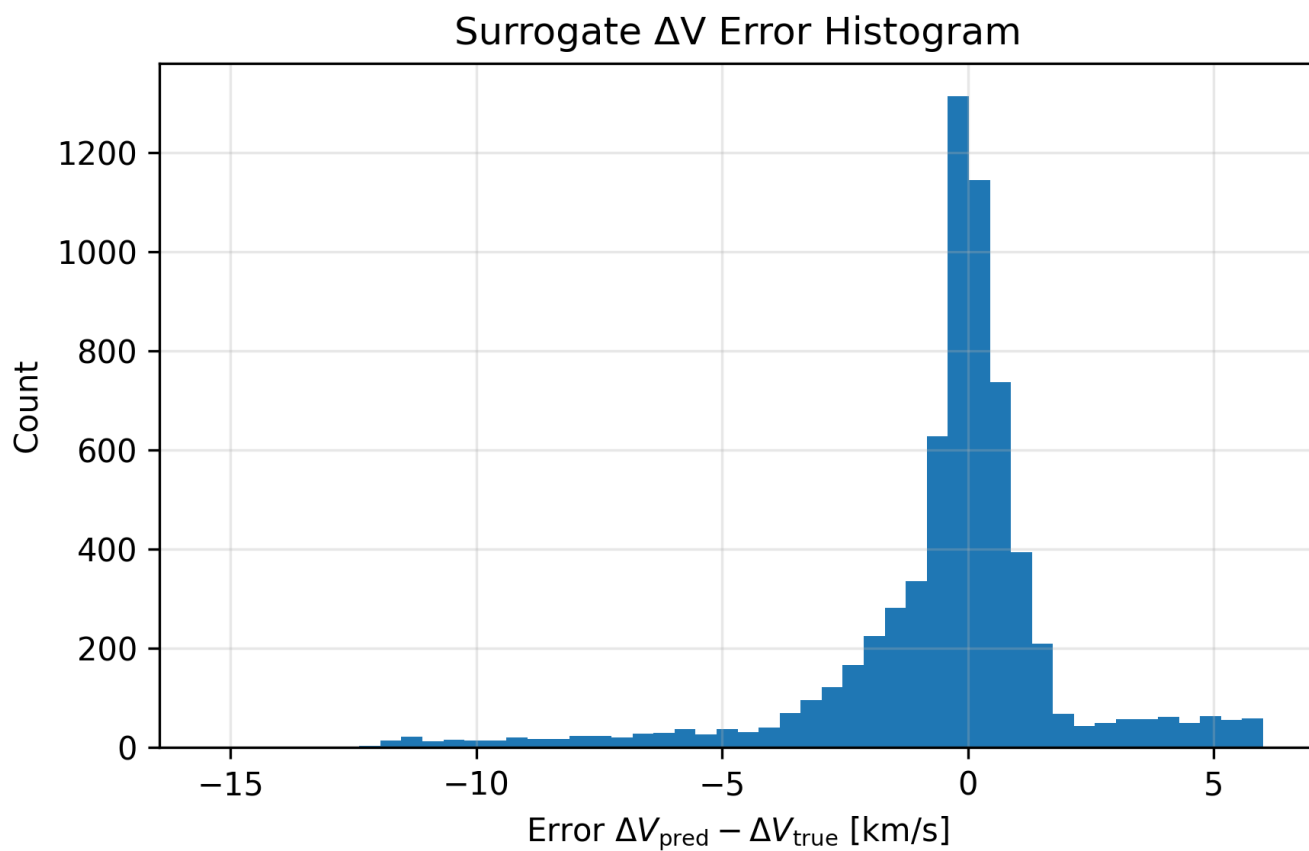


Fig. 25. Error histogram for 2038–2040.

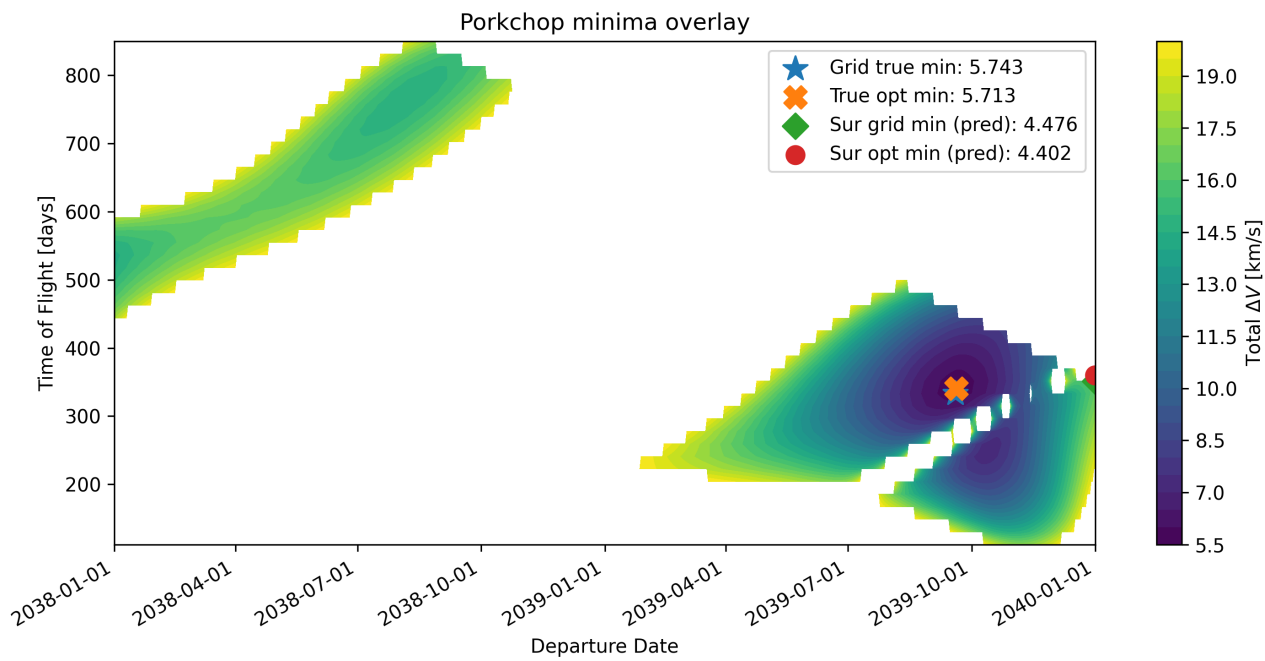


Fig. 26. Four-minima overlay for 2038–2040.

APPENDIX

```

1  import os
2  import sys
3  import atexit
4  from datetime import datetime
5  import pandas as pd
6  from tqdm import tqdm
7
8  from porkchop_run import (
9  generate_porkchop_data_daily,
10 plot_porkchop,
11 find_global_min_from_grid,
12 run_optimizer_with_history,
13 plot_porkchop_with_trajectory,
14 prepare_ml_npz_from_csv,
15 )
16
17 from train_surrogate import train_from_npz
18 from evaluate_surrogate import evaluate_surrogate
19
20
21 class Tee:
22     def __init__(self, *streams):
23         self.streams = streams
24
25     def write(self, data):
26         for s in self.streams:
27             try:
28                 s.write(data)
29                 s.flush()
30             except Exception:
31                 pass
32
33     def flush(self):
34         for s in self.streams:
35             try:
36                 s.flush()
37             except Exception:
38                 pass
39
40

```

```

41 def ensure_dir(path):
42     os.makedirs(path, exist_ok=True)
43
44
45 def combo_tag(dep_start, dep_end, arr_start, arr_end):
46     return f"dep{dep_start}_{dep_end}_arr{arr_start}_{arr_end}"
47
48
49 run_tag = datetime.now().strftime("%Y%m%d_%H%M%S")
50 base_out_dir = os.path.abspath(f"runs_{run_tag}")
51 ensure_dir(base_out_dir)
52
53 log_name = os.path.abspath(f"run_log_{run_tag}.txt")
54 log_file = open(log_name, "w", encoding="utf-8")
55
56 sys.stdout = Tee(sys.__stdout__, log_file)
57 sys.stderr = Tee(sys.__stderr__, log_file)
58
59 print(f"Logging console output to: {log_name}")
60
61
62 def _close_log():
63     try:
64         log_file.flush()
65         log_file.close()
66     except Exception:
67         pass
68
69
70 atexit.register(_close_log)
71
72
73 min_tof = 120.0
74 max_tof = 360.0
75 n_tof = 60
76 dv_cutoff = 20.0
77
78 years = [2030, 2032, 2034, 2036, 2038]
79
80 for y0 in tqdm(years, desc="Running porkchop + surrogate experiments", unit="window", ncols=100):
81     dep_start = f"{y0}-01-01"
82     dep_end = f"{y0 + 2}-01-01"
83     arr_start = f"{y0}-05-01"
84     arr_end = f"{y0 + 2}-12-26"
85
86     tag = combo_tag(dep_start, dep_end, arr_start, arr_end)
87     out_dir = os.path.abspath(os.path.join(base_out_dir, tag))
88     ensure_dir(out_dir)
89
90     print("\n" + "=" * 80)
91     print(f"RUN COMBO: {tag}")
92     print("=" * 80)
93
94     csv_name = f"porkchop_{tag}.csv"
95     clean_csv_name = f"porkchop_{tag}_clean.csv"
96     npz_name = f"ml_data_{tag}.npz"
97     norm_name = f"normalization_stats_{tag}.json"
98     model_name = f"dv_surrogate_model_{tag}.pt"
99
100     csv_path = os.path.join(out_dir, csv_name)
101     clean_csv_path = os.path.join(out_dir, clean_csv_name)
102     npz_path = os.path.join(out_dir, npz_name)
103     norm_path = os.path.join(out_dir, norm_name)
104     model_path = os.path.join(out_dir, model_name)
105
106     df, dep_times, tofs_days, dv_grid = generate_porkchop_data_daily(
107         dep_start=dep_start,
108         dep_end=dep_end,
109         arr_start=arr_start,
110         arr_end=arr_end,
111         min_tof_days=min_tof,
112         max_tof_days=max_tof,
113         n_tof=n_tof,
114         csv_path=csv_path,

```

```

115     dv_cutoff=dv_cutoff,
116     n_jobs=-1,
117 )
118
119     old_cwd = os.getcwd()
120     os.chdir(out_dir)
121     try:
122         plot_porkchop(dep_times, tofs_days, dv_grid, title_suffix=f"{dep_start}_{dep_end}")
123     finally:
124         os.chdir(old_cwd)
125
126     global_min = find_global_min_from_grid(dep_times, tofs_days, dv_grid)
127     print("GRID MIN")
128     print(global_min["dep_time"].iso)
129     print(f"{global_min['tof_days']:.6f}")
130     print(global_min["arr_time"].iso)
131     print(f"{global_min['dv_kms']:.6f}")
132
133     opt_result = run_optimizer_with_history(
134         dep_start=dep_start,
135         dep_end=dep_end,
136         min_tof_days=min_tof,
137         max_tof_days=max_tof,
138         dv_cutoff=dv_cutoff,
139         method="de",
140         seed=1,
141     )
142
143     print("OPT MIN")
144     print(opt_result["dep_time"].iso)
145     print(f"{opt_result['tof_days']:.6f}")
146     print(opt_result["arr_time"].iso)
147     print(f"{opt_result['dv_kms']:.6f}")
148     print(len(opt_result["history"]))
149
150     os.chdir(out_dir)
151     try:
152         plot_porkchop_with_trajectory(
153             dep_times,
154             tofs_days,
155             dv_grid,
156             global_min=global_min,
157             opt_result=opt_result,
158             title_suffix=f"{dep_start}_{dep_end}",
159         )
160     finally:
161         os.chdir(old_cwd)
162
163     if not os.path.exists(clean_csv_path):
164         df_raw = pd.read_csv(csv_path)
165         df_clean = df_raw.dropna(subset=["dv_total_kms"]).copy()
166         df_clean.to_csv(clean_csv_path, index=False)
167         print(f"Saved clean CSV: {clean_csv_path}")
168     else:
169         print(f"Clean CSV exists: {clean_csv_path}")
170
171     prepare_ml_npz_from_csv(csv_path, out_npz=npz_path, out_norm_json=norm_path)
172     train_from_npz(npz_path, save_path=model_path)
173
174     os.chdir(out_dir)
175     try:
176         evaluate_surrogate(
177             dep_start=dep_start,
178             dep_end=dep_end,
179             min_tof_days=min_tof,
180             max_tof_days=max_tof,
181             clean_csv_path=clean_csv_name,
182             model_checkpoint=model_name,
183             norm_stats_path=norm_name,
184             true_opt_result=opt_result,
185             dv_cutoff=dv_cutoff,
186             surrogate_opt_method="de",
187             surrogate_opt_seed=2,
188         )

```

```
189     finally:  
190         os.chdir(old_cwd)  
191  
192     print("nALL COMBINATIONS COMPLETE")  
193     print(f"Results saved under: {base_out_dir}")
```

Listing 1. Main Code

```

1  #!/usr/bin/env python
2  # -*- coding: utf-8 -*-
3
4  import os
5  import json
6  import argparse
7  import numpy as np
8  import pandas as pd
9
10 from astropy import units as u
11 from astropy.time import Time, TimeDelta
12
13 import matplotlib.pyplot as plt
14 import matplotlib.dates as mdates
15
16 from poliastro.bodies import Sun, Earth, Mars
17 from poliastro.twobody import Orbit
18 from poliastro.iod import izzo
19
20 from tqdm.auto import tqdm
21
22 try:
23     from joblib import Parallel, delayed
24     _HAS_JOBLIB = True
25 except Exception:
26     _HAS_JOBLIB = False
27
28 try:
29     from poliastro.ephem import Ephem
30     _HAS_EPHEM = True
31 except Exception:
32     _HAS_EPHEM = False
33
34 try:
35     from scipy.optimize import differential_evolution, minimize
36     _HAS SCIPY = True
37 except Exception:
38     _HAS SCIPY = False
39
40 DV_CUTOFF = 20.0
41 DATA_CSV_PATH = "porkchop_earth_mars_2030_2032_daily.csv"
42 N_JOBS = -1
43
44
45 def get_heliocentric_state(body, epoch):
46     orb = Orbit.from_body_ephem(body, epoch)
47     return orb.r, orb.v
48
49
50 def lambert_delta_v(r1, v1_planet, r2, v2_planet, tof_seconds):
51     (v1_trans, v2_trans), = izzo.lambert(Sun.k, r1, r2, tof_seconds)
52     dv1_vec = v1_trans - v1_planet
53     dv2_vec = v2_trans - v2_planet
54     dv1 = np.linalg.norm(dv1_vec.to(u.km / u.s).value)
55     dv2 = np.linalg.norm(dv2_vec.to(u.km / u.s).value)
56     return dv1 + dv2, dv1, dv2
57
58
59 def delta_v_objective(dep_time, tof_days, dv_cutoff=DV_CUTOFF, return_components=False):
60     try:
61         tof_days = float(tof_days)
62         if tof_days <= 0.0:
63             return (np.nan, np.nan, np.nan) if return_components else np.nan
64
65         tof_seconds = (tof_days * u.day).to(u.s)
66         r_E, v_E = get_heliocentric_state(Earth, dep_time)
67         arr_time = dep_time + TimeDelta(tof_days * u.day)
68         r_M, v_M = get_heliocentric_state(Mars, arr_time)
69         dv_total, dv1, dv2 = lambert_delta_v(r_E, v_E, r_M, v_M, tof_seconds)
70
71         if (not np.isfinite(dv_total)) or (dv_total > dv_cutoff):
72             return (np.nan, np.nan, np.nan) if return_components else np.nan
73

```

```

74     return (dv_total, dv1, dv2) if return_components else dv_total
75
76     except Exception:
77     return (np.nan, np.nan, np.nan) if return_components else np.nan
78
79
80     def _mars_states_vectorized(arr_times):
81     if _HAS_EPHEM:
82     try:
83     eph = Ephem.from_body(Mars, arr_times)
84     r_M, v_M = eph.rv(arr_times)
85     r_list = [r_M[i] for i in range(len(arr_times))]
86     v_list = [v_M[i] for i in range(len(arr_times))]
87     return r_list, v_list
88     except Exception:
89     pass
90
91     r_list, v_list = [], []
92     for t in arr_times:
93     r, v = get_heliocentric_state(Mars, t)
94     r_list.append(r)
95     v_list.append(v)
96     return r_list, v_list
97
98
99     def _compute_one_departure_fast(t_dep, tofs_days, dv_cutoff):
100     n_tof = len(tofs_days)
101
102     dv_row = np.full(n_tof, np.nan, dtype=np.float64)
103     dv1_row = np.full(n_tof, np.nan, dtype=np.float64)
104     dv2_row = np.full(n_tof, np.nan, dtype=np.float64)
105
106     r_E, v_E = get_heliocentric_state(Earth, t_dep)
107
108     arr_times = t_dep + TimeDelta(tofs_days * u.day)
109     arr_tdb = arr_times.tdb
110
111     arr_jd_row = arr_tdb.jd.astype(np.float64)
112     arr_iso_row = arr_tdb.iso.astype(object)
113
114     r_M_list, v_M_list = _mars_states_vectorized(arr_times)
115
116     for j, tof_days in enumerate(tofs_days):
117     try:
118     tof_seconds = (float(tof_days) * u.day).to(u.s)
119     dv_total, dv1, dv2 = lambert_delta_v(r_E, v_E, r_M_list[j], v_M_list[j], tof_seconds)
120
121     if (not np.isfinite(dv_total)) or (dv_total > dv_cutoff):
122     continue
123
124     dv_row[j] = dv_total
125     dv1_row[j] = dv1
126     dv2_row[j] = dv2
127     except Exception:
128     continue
129
130     return dv_row, dv1_row, dv2_row, arr_jd_row, arr_iso_row
131
132
133     def generate_porkchop_data_daily(
134     dep_start="2030-01-01",
135     dep_end="2032-01-01",
136     arr_start=None,
137     arr_end=None,
138     departure_step_days=1,
139     min_tof_days=100.0,
140     max_tof_days=400.0,
141     n_tof=60,
142     csv_path=DATA_CSV_PATH,
143     dv_cutoff=DV_CUTOFF,
144     n_jobs=N_JOBS,
145     ):
146     t_dep_start = Time(dep_start, scale="tdb").tdb
147     t_dep_end = Time(dep_end, scale="tdb").tdb

```

```

148
149 if arr_start is not None and arr_end is not None:
150     t_arr_start = Time(arr_start, scale="tdb").tdb
151     t_arr_end = Time(arr_end, scale="tdb").tdb
152     min_tof = (t_arr_start - t_dep_end).to(u.day).value
153     max_tof = (t_arr_end - t_dep_start).to(u.day).value
154     min_tof_days = float(max(min_tof, 1.0))
155     max_tof_days = float(max(max_tof, min_tof_days + 1.0))
156     print(f"Derived TOF range: [{min_tof_days:.3f}, {max_tof_days:.3f}] days")
157
158     total_days = int((t_dep_end - t_dep_start).to(u.day).value)
159     day_offsets = np.arange(0, total_days + 1, int(departure_step_days), dtype=int)
160     dep_times = (t_dep_start + TimeDelta(day_offsets * u.day)).tdb
161     n_departures = len(dep_times)
162
163     tofs_days = np.linspace(float(min_tof_days), float(max_tof_days), int(n_tof), dtype=np.float64)
164     n_tof = len(tofs_days)
165
166     print(f"Grid: {n_departures} departures {n_tof} TOFs")
167     print(f"Departure window: {t_dep_start.iso} to {t_dep_end.iso}")
168     if arr_start is not None and arr_end is not None:
169         print(f"Arrival window: {Time(arr_start, scale='tdb').tdb.iso} to {Time(arr_end, scale='tdb').tdb.iso}")
170
171     print(f"DV cutoff: {dv_cutoff} km/s")
172     if not _HAS_EPHEM:
173         print("Ephem unavailable; Mars ephemeris will be slower.")
174
175     dep_jd = dep_times.jd.astype(np.float64)
176     dep_iso = dep_times.iso.astype(object)
177
178     dv_grid = np.full((n_departures, n_tof), np.nan, dtype=np.float64)
179     dv1_grid = np.full((n_departures, n_tof), np.nan, dtype=np.float64)
180     dv2_grid = np.full((n_departures, n_tof), np.nan, dtype=np.float64)
181
182     arr_jd_grid = np.empty((n_departures, n_tof), dtype=np.float64)
183     arr_iso_grid = np.empty((n_departures, n_tof), dtype=object)
184
185     do_parallel = _HAS_JOBLIB and isinstance(n_jobs, int) and (n_jobs != 1) and (n_departures > 1)
186
187     if do_parallel:
188         results = Parallel(n_jobs=n_jobs, prefer="processes")(
189             delayed(_compute_one_departure_fast)(dep_times[i], tofs_days, dv_cutoff)
190             for i in range(n_departures)
191         )
192         for i, (dv_row, dv1_row, dv2_row, arr_jd_row, arr_iso_row) in enumerate(results):
193             dv_grid[i, :] = dv_row
194             dv1_grid[i, :] = dv1_row
195             dv2_grid[i, :] = dv2_row
196             arr_jd_grid[i, :] = arr_jd_row
197             arr_iso_grid[i, :] = arr_iso_row
198         else:
199             for i, t_dep in enumerate(tqdm(dep_times, desc="Departures", unit="dep")):
200                 dv_row, dv1_row, dv2_row, arr_jd_row, arr_iso_row = _compute_one_departure_fast(
201                     t_dep, tofs_days, dv_cutoff
202                 )
203                 dv_grid[i, :] = dv_row
204                 dv1_grid[i, :] = dv1_row
205                 dv2_grid[i, :] = dv2_row
206                 arr_jd_grid[i, :] = arr_jd_row
207                 arr_iso_grid[i, :] = arr_iso_row
208
209     n_total = n_departures * n_tof
210     dep_jd_flat = np.repeat(dep_jd, n_tof)
211     dep_iso_flat = np.repeat(dep_iso, n_tof)
212     tof_flat = np.tile(tofs_days, n_departures)
213
214     df = pd.DataFrame(
215         {
216             "dep_julian": dep_jd_flat,
217             "arr_julian": arr_jd_grid.reshape(n_total),
218             "dep_iso": dep_iso_flat,
219             "arr_iso": arr_iso_grid.reshape(n_total),
220             "tof_days": tof_flat.astype(np.float64),
221             "dv_total_kms": dv_grid.reshape(n_total),

```

```

221     "dv_dep_kms": dv1_grid.reshape(n_total),
222     "dv_arr_kms": dv2_grid.reshape(n_total),
223 }
224 )
225
226 if arr_start is not None and arr_end is not None:
227     t_arr_start = Time(arr_start, scale="tdb").tdb
228     t_arr_end = Time(arr_end, scale="tdb").tdb
229     arr_t = Time(df["arr_julian"].values.astype(np.float64), format="jd", scale="tdb").tdb
230     in_window = (arr_t.jd >= t_arr_start.jd) & (arr_t.jd <= t_arr_end.jd)
231     df.loc[~in_window, ["dv_total_kms", "dv_dep_kms", "dv_arr_kms"]] = np.nan
232
233 df.to_csv(csv_path, index=False)
234 print(f"Saved CSV: {csv_path} ({len(df)} rows)")
235 return df, dep_times, tofs_days, dv_grid
236
237
238 def plot_porkchop(dep_times, tofs_days, dv_grid, title_suffix="run"):
239     os.makedirs("plots", exist_ok=True)
240
241     dep_datetimes = dep_times.to_datetime()
242     dep_mpl = mdates.date2num(dep_datetimes)
243
244     D_dep, D_tof = np.meshgrid(dep_mpl, tofs_days, indexing="ij")
245     Z = np.ma.masked_invalid(dv_grid)
246
247     plt.figure(figsize=(10, 5))
248     cs = plt.contourf(D_dep, D_tof, Z, levels=30)
249     cbar = plt.colorbar(cs)
250     cbar.set_label(r"Total $\Delta V$ [km/s]")
251
252     try:
253         cs_lines = plt.contour(D_dep, D_tof, Z, levels=10, colors="k", linewidths=0.5)
254         plt.clabel(cs_lines, inline=True, fontsize=8, fmt="%.1f")
255     except Exception:
256         pass
257
258     ax = plt.gca()
259     ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m-%d"))
260     ax.xaxis.set_major_locator(mdates.AutoDateLocator())
261
262     plt.xticks(rotation=30, ha="right")
263     plt.xlabel("Departure Date")
264     plt.ylabel("Time of Flight [days]")
265     plt.title(rf"EarthMars Pork-Chop (Total $\Delta V$) {title_suffix}")
266
267     plt.tight_layout()
268     save_path = f"plots/porkchop_{title_suffix}.png"
269     plt.savefig(save_path, dpi=300)
270     print(f"Saved plot: {save_path}")
271
272     def rebuild_grid_from_csv(df):
273         dep_julians = np.sort(df["dep_julian"].unique())
274         tofs_days = np.sort(df["tof_days"].unique())
275         dep_times = Time(dep_julians, format="jd", scale="tdb").tdb
276
277         grid = (
278             df.pivot(index="dep_julian", columns="tof_days", values="dv_total_kms")
279             .reindex(index=dep_julians, columns=tofs_days)
280             .to_numpy()
281         )
282         return dep_times, tofs_days, grid
283
284
285     def find_global_min_from_grid(dep_times, tofs_days, dv_grid):
286         Z = np.array(dv_grid, dtype=np.float64)
287         if not np.isfinite(Z).any():
288             raise RuntimeError("dv_grid has no finite values")
289         idx = np.nanargmin(Z)
290         i, j = np.unravel_index(idx, Z.shape)
291         dep_time = dep_times[i].tdb
292         tof = float(tofs_days[j])
293         arr_time = (dep_time + TimeDelta(tof * u.day)).tdb
294         dv = float(Z[i, j])

```

```

295     return {"i": int(i), "j": int(j), "dep_time": dep_time, "tof_days": tof, "arr_time": arr_time, "dv_kms"
296           : dv}
297
298     def _continuous_objective_factory(dv_cutoff, dep_jd_bounds, tof_bounds):
299         dep_lo, dep_hi = dep_jd_bounds
300         tof_lo, tof_hi = tof_bounds
301         BIG = 1e6
302
303         def f(x):
304             dep_jd = float(x[0])
305             tof_days = float(x[1])
306
307             if dep_jd < dep_lo or dep_jd > dep_hi or tof_days < tof_lo or tof_days > tof_hi:
308                 return BIG
309
310             dep_time = Time(dep_jd, format="jd", scale="tdb").tdb
311             dv = delta_v_objective(dep_time, tof_days, dv_cutoff=dv_cutoff, return_components=False)
312             if (dv is None) or (not np.isfinite(dv)):
313                 return BIG
314             return float(dv)
315
316         return f
317
318
319     def run_optimizer_with_history(
320         dep_start, dep_end,
321         min_tof_days, max_tof_days,
322         dv_cutoff=DV_CUTOFF,
323         method="de",
324         seed=1,
325     ):
326         if not _HAS SCIPY:
327             raise RuntimeError("SciPy not available")
328
329         t0 = Time(dep_start, scale="tdb").tdb
330         t1 = Time(dep_end, scale="tdb").tdb
331
332         dep_bounds = (float(t0.jd), float(t1.jd))
333         tof_bounds = (float(min_tof_days), float(max_tof_days))
334
335         f = _continuous_objective_factory(dv_cutoff, dep_bounds, tof_bounds)
336         history = []
337
338         x0 = np.array([(dep_bounds[0] + dep_bounds[1]) / 2.0, (tof_bounds[0] + tof_bounds[1]) / 2.0], dtype=
339                       float)
340
341         if method.lower() == "de":
342             bounds = [dep_bounds, tof_bounds]
343
344             def cb(xk, convergence=None):
345                 history.append((float(xk[0]), float(xk[1]), float(f(xk))))
346                 return False
347
348             res = differential_evolution(
349                 f,
350                 bounds=bounds,
351                 seed=seed,
352                 callback=cb,
353                 polish=True,
354                 updating="deferred",
355                 workers=1,
356             )
357             xbest = res.x
358         else:
359             bounds = [dep_bounds, tof_bounds]
360
361             def cb(xk):
362                 history.append((float(xk[0]), float(xk[1]), float(f(xk))))
363
364             res = minimize(
365                 f,
366                 x0=x0,
367                 method="L-BFGS-B",

```

```

367     bounds=bounds,
368     callback=cb,
369 )
370     xbest = res.x
371
372     dep_jd_best = float(xbest[0])
373     tof_best = float(xbest[1])
374     dep_time_best = Time(dep_jd_best, format="jd", scale="tdb").tdb
375     arr_time_best = (dep_time_best + TimeDelta(tof_best * u.day)).tdb
376     dv_best = float(delta_v_objective(dep_time_best, tof_best, dv_cutoff=dv_cutoff))
377
378     return {
379         "method": method,
380         "success": bool(getattr(res, "success", True)),
381         "message": str(getattr(res, "message", "")),
382         "nfev": int(getattr(res, "nfev", -1)) if getattr(res, "nfev", None) is not None else -1,
383         "dep_time": dep_time_best,
384         "tof_days": tof_best,
385         "arr_time": arr_time_best,
386         "dv_kms": dv_best,
387         "history": history,
388     }
389
390
391     def plot_porkchop_with_trajectory(dep_times, tofs_days, dv_grid, global_min, opt_result, title_suffix="
    opt"):
392         os.makedirs("plots", exist_ok=True)
393
394         dep_datetimes = dep_times.to_datetime()
395         dep_mpl = mdates.date2num(dep_datetimes)
396
397         D_dep, D_tof = np.meshgrid(dep_mpl, tofs_days, indexing="ij")
398         Z = np.ma.masked_invalid(dv_grid)
399
400         plt.figure(figsize=(10, 5))
401         cs = plt.contourf(D_dep, D_tof, Z, levels=30)
402         cbar = plt.colorbar(cs)
403         cbar.set_label(r"Total $\Delta V$ [km/s]")
404
405         ax = plt.gca()
406         ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m-%d"))
407         ax.xaxis.set_major_locator(mdates.AutoDateLocator())
408         plt.xticks(rotation=30, ha="right")
409         plt.xlabel("Departure Date")
410         plt.ylabel("Time of Flight [days]")
411
412         g_dep = mdates.date2num(global_min["dep_time"].to_datetime())
413         g_tof = float(global_min["tof_days"])
414         plt.scatter([g_dep], [g_tof], marker="*", s=180, label=f"Grid min: {global_min['dv_kms']:.3f} km/s")
415
416         hist = opt_result.get("history", [])
417         if len(hist) > 0:
418             hist_dep_dt = [Time(h[0], format="jd", scale="tdb").to_datetime() for h in hist]
419             hist_dep_mpl = mdates.date2num(hist_dep_dt)
420             hist_tof = [h[1] for h in hist]
421             plt.plot(hist_dep_mpl, hist_tof, linewidth=2, label=f"Opt path ({opt_result['method']})")
422             plt.scatter([hist_dep_mpl[-1]], [hist_tof[-1]], marker="o", s=60)
423
424             o_dep = mdates.date2num(opt_result["dep_time"].to_datetime())
425             o_tof = float(opt_result["tof_days"])
426             plt.scatter([o_dep], [o_tof], marker="X", s=120, label=f"Opt min: {opt_result['dv_kms']:.3f} km/s")
427
428             plt.title(rf"Pork-Chop + Opt Path ({title_suffix})")
429             plt.legend(loc="best")
430             plt.tight_layout()
431
432             save_path = f"plots/porkchop_{title_suffix}_optpath.png"
433             plt.savefig(save_path, dpi=300)
434             print(f"Saved plot: {save_path}")
435
436         def prepare_ml_npz_from_csv(csv_path, out_npz="ml_data_all.npz", out_norm_json="normalization_stats.
            json"):
437             df = pd.read_csv(csv_path)
438             df_clean = df.dropna(subset=["dv_total_kms"]).copy()

```

```

439
440 X = df_clean[["dep_julian", "tof_days"]].values.astype(np.float32)
441 y = df_clean["dv_total_kms"].values.astype(np.float32).reshape(-1, 1)
442
443 dep_julian = df_clean["dep_julian"].values.astype(np.float64)
444 dep_year = df_clean["dep_iso"].str.slice(0, 4).astype(np.int16).values
445
446 X_mean = X.mean(axis=0)
447 X_std = X.std(axis=0)
448 X_std[X_std == 0.0] = 1.0
449 X_norm = (X - X_mean) / X_std
450
451 with open(out_norm_json, "w") as f:
452     json.dump({"X_mean": X_mean.tolist(), "X_std": X_std.tolist()}, f)
453
454 np.savez(
455     out_npz,
456     X_norm=X_norm,
457     y=y,
458     dep_julian=dep_julian,
459     dep_year=dep_year,
460 )
461
462 print(f"Saved: {out_norm_json}")
463 print(f"Saved: {out_npz}")
464 print(f"Rows: {len(df_clean)} / {len(df)}")
465 return out_npz
466
467
468 def parse_args():
469     p = argparse.ArgumentParser(description="Earth->Mars porkchop generator")
470
471     p.add_argument("--dep-start", type=str, default="2030-01-01")
472     p.add_argument("--dep-end", type=str, default="2032-01-01")
473     p.add_argument("--arr-start", type=str, default=None)
474     p.add_argument("--arr-end", type=str, default=None)
475
476     p.add_argument("--dep-step", type=int, default=1)
477     p.add_argument("--min-tof", type=float, default=120.0)
478     p.add_argument("--max-tof", type=float, default=360.0)
479     p.add_argument("--n-tof", type=int, default=60)
480
481     p.add_argument("--dv-cutoff", type=float, default=DV_CUTOFF)
482     p.add_argument("--csv", type=str, default=DATA_CSV_PATH)
483     p.add_argument("--n-jobs", type=int, default=N_JOBS)
484
485     p.add_argument("--no-generate", action="store_true")
486     p.add_argument("--plot", action="store_true")
487     p.add_argument("--opt-plot", action="store_true")
488     p.add_argument("--opt-method", type=str, default="de", choices=["de", "lbfgsb"])
489     p.add_argument("--opt-seed", type=int, default=1)
490     p.add_argument("--prep-ml", action="store_true")
491     p.add_argument("--title", type=str, default="run")
492
493     return p.parse_args()
494
495
496 if __name__ == "__main__":
497     args = parse_args()
498
499     dep_times = None
500     tofs_days = None
501     dv_grid = None
502
503     if not args.no_generate:
504         _, dep_times, tofs_days, dv_grid = generate_porkchop_data_daily(
505             dep_start=args.dep_start,
506             dep_end=args.dep_end,
507             arr_start=args.arr_start,
508             arr_end=args.arr_end,
509             departure_step_days=args.dep_step,
510             min_tof_days=args.min_tof,
511             max_tof_days=args.max_tof,
512             n_tof=args.n_tof,

```

```

513 csv_path=args.csv,
514 dv_cutoff=args.dv_cutoff,
515 n_jobs=args.n_jobs,
516 )
517
518 if args.plot:
519     if dep_times is None or tofs_days is None or dv_grid is None:
520         df_plot = pd.read_csv(args.csv)
521         dep_times, tofs_days, dv_grid = rebuild_grid_from_csv(df_plot)
522         plot_porkchop(dep_times, tofs_days, dv_grid, title_suffix=args.title)
523
524 if args.opt_plot:
525     if dep_times is None or tofs_days is None or dv_grid is None:
526         df_plot = pd.read_csv(args.csv)
527         dep_times, tofs_days, dv_grid = rebuild_grid_from_csv(df_plot)
528
529     global_min = find_global_min_from_grid(dep_times, tofs_days, dv_grid)
530     print(f"Grid min dep: {global_min['dep_time'].iso}")
531     print(f"Grid min tof: {global_min['tof_days']:.6f}")
532     print(f"Grid min arr: {global_min['arr_time'].iso}")
533     print(f"Grid min dv: {global_min['dv_kms']:.6f}")
534
535     opt_result = run_optimizer_with_history(
536         dep_start=args.dep_start,
537         dep_end=args.dep_end,
538         min_tof_days=args.min_tof,
539         max_tof_days=args.max_tof,
540         dv_cutoff=args.dv_cutoff,
541         method=args.opt_method,
542         seed=args.opt_seed,
543     )
544
545     print(f"Opt min dep: {opt_result['dep_time'].iso}")
546     print(f"Opt min tof: {opt_result['tof_days']:.6f}")
547     print(f"Opt min arr: {opt_result['arr_time'].iso}")
548     print(f"Opt min dv: {opt_result['dv_kms']:.6f}")
549     print(f"History pts: {len(opt_result['history'])}")
550
551     plot_porkchop_with_trajectory(
552         dep_times, tofs_days, dv_grid,
553         global_min=global_min,
554         opt_result=opt_result,
555         title_suffix=args.title,
556     )
557
558 if args.prep_ml:
559     prepare_ml_npz_from_csv(args.csv, out_npz="ml_data_all.npz", out_norm_json="normalization_stats.json")

```

Listing 2. Data Generation

```

1  import json
2  import random
3  import numpy as np
4  import torch
5  import torch.nn as nn
6  import torch.optim as optim
7  from torch.utils.data import TensorDataset, DataLoader
8
9  DATA_NPZ_PATH = "ml_data_all.npz"
10
11  N_EPOCHS = 200
12  BATCH_SIZE = 128
13  LR = 3e-4
14  PATIENCE = 20
15
16  DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
17
18
19  def load_ml_data(npz_path=DATA_NPZ_PATH):
20  data = np.load(npz_path)
21  X_norm = data["X_norm"]
22  y = data["y"]
23  dep_j = data["dep_julian"]
24  dep_year = data["dep_year"]
25
26  idx = np.argsort(dep_j)
27  X_norm = X_norm[idx]
28  y = y[idx]
29  dep_j = dep_j[idx]
30  dep_year = dep_year[idx]
31
32  return X_norm, y, dep_j, dep_year
33
34
35  def train_val_test_split(X, y, dep_j, train_frac=0.7, val_frac=0.15):
36  N = len(dep_j)
37  n_train = int(train_frac * N)
38  n_val = int(val_frac * N)
39
40  X_train = X[:n_train]
41  y_train = y[:n_train]
42
43  X_val = X[n_train:n_train + n_val]
44  y_val = y[n_train:n_train + n_val]
45
46  X_test = X[n_train + n_val:]
47  y_test = y[n_train + n_val:]
48
49  dep_train = dep_j[:n_train]
50  dep_val = dep_j[n_train:n_train + n_val]
51  dep_test = dep_j[n_train + n_val:]
52
53  print("Split sizes:")
54  print(f"  Train: {X_train.shape[0]}")
55  print(f"  Val:   {X_val.shape[0]}")
56  print(f"  Test:  {X_test.shape[0]}")
57
58  return (
59  X_train, y_train, dep_train,
60  X_val, y_val, dep_val,
61  X_test, y_test, dep_test,
62  )
63
64
65  def make_loaders(X_train, y_train, X_val, y_val, batch_size=BATCH_SIZE):
66  X_train_t = torch.from_numpy(X_train).float()
67  y_train_t = torch.from_numpy(y_train).float()
68
69  X_val_t = torch.from_numpy(X_val).float()
70  y_val_t = torch.from_numpy(y_val).float()
71
72  train_ds = TensorDataset(X_train_t, y_train_t)
73  val_ds = TensorDataset(X_val_t, y_val_t)

```

```

74
75 train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
76 val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False)
77
78 return train_loader, val_loader
79
80
81 class DVSSurrogate(nn.Module):
82     def __init__(self, in_dim=2, hidden_dim=128, dropout_p=0.1):
83         super().__init__()
84         self.net = nn.Sequential(
85             nn.Linear(in_dim, hidden_dim),
86             nn.ReLU(),
87             nn.Dropout(dropout_p),
88             nn.Linear(hidden_dim, hidden_dim),
89             nn.ReLU(),
90             nn.Dropout(dropout_p),
91             nn.Linear(hidden_dim, hidden_dim),
92             nn.ReLU(),
93             nn.Linear(hidden_dim, 1),
94         )
95
96     def forward(self, x):
97         return self.net(x)
98
99
100     def mae(y_true, y_pred):
101         return torch.mean(torch.abs(y_true - y_pred))
102
103
104     def rmse(y_true, y_pred):
105         return torch.sqrt(torch.mean((y_true - y_pred) ** 2))
106
107
108     def r2_score(y_true, y_pred):
109         y_mean = torch.mean(y_true)
110         ss_tot = torch.sum((y_true - y_mean) ** 2)
111         ss_res = torch.sum((y_true - y_pred) ** 2)
112         return 1.0 - ss_res / ss_tot
113
114
115     def train_model(model, train_loader, val_loader, n_epochs=N_EPOCHS, lr=LR, device=DEVICE, patience=
        PATIENCE):
116         model.to(device)
117         criterion = nn.MSELoss()
118         optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)
119         scheduler = optim.lr_scheduler.ReduceLROnPlateau(
120             optimizer, mode="min", factor=0.5, patience=5, min_lr=1e-6
121         )
122
123         best_val_loss = float("inf")
124         best_state_dict = None
125         epochs_no_improve = 0
126
127         for epoch in range(1, n_epochs + 1):
128             model.train()
129             train_loss = 0.0
130             for Xb, yb in train_loader:
131                 Xb = Xb.to(device)
132                 yb = yb.to(device)
133
134                 optimizer.zero_grad()
135                 preds = model(Xb)
136                 loss = criterion(preds, yb)
137                 loss.backward()
138                 torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
139                 optimizer.step()
140                 train_loss += loss.item() * Xb.size(0)
141
142             train_loss /= len(train_loader.dataset)
143
144             model.eval()
145             val_loss = 0.0
146             val_mae = 0.0

```

```

147     val_rmse = 0.0
148     val_r2 = 0.0
149     n_val = 0
150
151     with torch.no_grad():
152         for Xb, yb in val_loader:
153             Xb = Xb.to(device)
154             yb = yb.to(device)
155
156             preds = model(Xb)
157             loss = criterion(preds, yb)
158
159             bs = Xb.size(0)
160             val_loss += loss.item() * bs
161             val_mae += mae(yb, preds).item() * bs
162             val_rmse += rmse(yb, preds).item() * bs
163             val_r2 += r2_score(yb, preds).item() * bs
164             n_val += bs
165
166         val_loss /= n_val
167         val_mae /= n_val
168         val_rmse /= n_val
169         val_r2 /= n_val
170
171         scheduler.step(val_loss)
172
173     if epoch % 5 == 0 or epoch == 1:
174         current_lr = optimizer.param_groups[0]["lr"]
175         print(
176             f"Epoch {epoch:03d}: "
177             f"train MSE={train_loss:.4f}, "
178             f"val MSE={val_loss:.4f}, "
179             f"val MAE={val_mae:.4f}, "
180             f"val RMSE={val_rmse:.4f}, "
181             f"val R2={val_r2:.4f}, "
182             f"LR={current_lr:.2e}"
183         )
184
185     if val_loss < best_val_loss - 1e-4:
186         best_val_loss = val_loss
187         best_state_dict = model.state_dict()
188         epochs_no_improve = 0
189     else:
190         epochs_no_improve += 1
191     if epochs_no_improve >= patience:
192         print(f"Early stopping at epoch {epoch}")
193         break
194
195     if best_state_dict is not None:
196         model.load_state_dict(best_state_dict)
197
198     return model
199
200
201 def evaluate_on_test(model, X_test, y_test, label="Test", device=DEVICE, y_mean=None, y_std=None):
202     model.to(device)
203     model.eval()
204
205     X_t = torch.from_numpy(X_test).float().to(device)
206     y_t = torch.from_numpy(y_test).float().to(device)
207
208     with torch.no_grad():
209         preds = model(X_t)
210
211     if (y_mean is not None) and (y_std is not None):
212         preds_denorm = preds * y_std + y_mean
213         y_denorm = y_t * y_std + y_mean
214     else:
215         preds_denorm = preds
216         y_denorm = y_t
217
218     test_mae = mae(y_denorm, preds_denorm).item()
219     test_rmse = rmse(y_denorm, preds_denorm).item()
220     test_r2 = r2_score(y_denorm, preds_denorm).item()

```

```

221
222 print(f"{label} performance:")
223 print(f"   MAE   = {test_mae:.4f} km/s")
224 print(f"   RMSE  = {test_rmse:.4f} km/s")
225 print(f"   R2    = {test_r2:.4f}")
226
227 return preds_denorm.cpu().numpy()
228
229
230 def train_from_npz(npz_path=DATA_NPZ_PATH, save_path="dv_surrogate_model_2030_2032_supervised.pt"):
231 print(f"Using device: {DEVICE}")
232
233 torch.manual_seed(42)
234 np.random.seed(42)
235 random.seed(42)
236 if torch.cuda.is_available():
237     torch.cuda.manual_seed_all(42)
238
239 X_norm, y, dep_j, _ = load_ml_data(npz_path)
240 print(f"Total samples: {X_norm.shape[0]}")
241
242 y_mean = y.mean()
243 y_std = y.std()
244 y_norm = (y - y_mean) / y_std
245
246 (
247     X_train, y_train, _,
248     X_val, y_val, _,
249     X_test, y_test, _,
250 ) = train_val_test_split(X_norm, y_norm, dep_j, train_frac=0.7, val_frac=0.15)
251
252 train_loader, val_loader = make_loaders(X_train, y_train, X_val, y_val, batch_size=BATCH_SIZE)
253
254 in_dim = X_norm.shape[1]
255 model = DVSSurrogate(in_dim=in_dim, hidden_dim=128, dropout_p=0.1)
256
257 model = train_model(model, train_loader, val_loader, n_epochs=N_EPOCHS, lr=LR, device=DEVICE, patience=
    PATIENCE)
258
259 evaluate_on_test(model, X_val, y_val, label="Validation", device=DEVICE, y_mean=y_mean, y_std=y_std)
260 evaluate_on_test(model, X_test, y_test, label="Test", device=DEVICE, y_mean=y_mean, y_std=y_std)
261 evaluate_on_test(model, X_norm, y_norm, label="All", device=DEVICE, y_mean=y_mean, y_std=y_std)
262
263 torch.save(
264     {
265         "model_state_dict": model.state_dict(),
266         "y_mean": float(y_mean),
267         "y_std": float(y_std),
268         "in_dim": in_dim,
269         "hidden_dim": 128,
270         "dropout_p": 0.1,
271     },
272     save_path,
273 )
274 print(f"Saved model: {save_path}")
275 return model
276
277
278 if __name__ == "__main__":
279     train_from_npz(DATA_NPZ_PATH)

```

Listing 3. Train Data

```

1  import os
2  import json
3  import numpy as np
4  import pandas as pd
5  import matplotlib.pyplot as plt
6  import matplotlib.dates as mdates
7  from astropy.time import Time, TimeDelta
8  from astropy import units as u
9
10 import torch
11 import torch.nn as nn
12
13 from porkchop_run import delta_v_objective
14
15 try:
16 from scipy.optimize import differential_evolution, minimize
17 _HAS SCIPY = True
18 except Exception:
19 _HAS SCIPY = False
20
21
22 CLEAN_CSV_PATH = "porkchop_earth_mars_2030_2032_daily_clean.csv"
23 MODEL_CHECKPOINT = "dv_surrogate_model_2030_2032_supervised.pt"
24 NORM_STATS_PATH = "normalization_stats.json"
25
26
27 class DVSurrogate(nn.Module):
28 def __init__(self, in_dim=2, hidden_dim=128, dropout_p=0.1):
29 super().__init__()
30 self.net = nn.Sequential(
31 nn.Linear(in_dim, hidden_dim),
32 nn.ReLU(),
33 nn.Dropout(dropout_p),
34 nn.Linear(hidden_dim, hidden_dim),
35 nn.ReLU(),
36 nn.Dropout(dropout_p),
37 nn.Linear(hidden_dim, hidden_dim),
38 nn.ReLU(),
39 nn.Linear(hidden_dim, 1),
40 )
41
42 def forward(self, x):
43 return self.net(x)
44
45
46 def _window_tag(dep_start, dep_end, min_tof_days, max_tof_days):
47 t_dep0 = Time(dep_start, scale="tdb").tdb
48 t_dep1 = Time(dep_end, scale="tdb").tdb
49 t_arr0 = (t_dep0 + TimeDelta(float(min_tof_days) * u.day)).tdb
50 t_arr1 = (t_dep1 + TimeDelta(float(max_tof_days) * u.day)).tdb
51
52 dep0 = t_dep0.to_datetime().strftime("%Y%m%d")
53 dep1 = t_dep1.to_datetime().strftime("%Y%m%d")
54 arr0 = t_arr0.to_datetime().strftime("%Y%m%d")
55 arr1 = t_arr1.to_datetime().strftime("%Y%m%d")
56
57 return f"dep{dep0}-{dep1}_arr{arr0}-{arr1}"
58
59
60 def rebuild_grid_from_df(df, dep_decimals=9, tof_decimals=6):
61 dep_julians = np.sort(df["dep_julian"].unique()).astype(np.float64)
62 tofs_days = np.sort(df["tof_days"].unique()).astype(np.float64)
63 dep_times = Time(dep_julians, format="jd", scale="tdb").tdb
64
65 dep_keys = np.round(dep_julians, dep_decimals)
66 tof_keys = np.round(tofs_days, tof_decimals)
67
68 dep_index = {float(k): i for i, k in enumerate(dep_keys)}
69 tof_index = {float(k): j for j, k in enumerate(tof_keys)}
70
71 grid = (
72 df.assign(
73 dep_key=np.round(df["dep_julian"].astype(np.float64), dep_decimals),

```

```

74     tof_key=np.round(df["tof_days"].astype(np.float64), tof_decimals),
75     )
76     .pivot(index="dep_key", columns="tof_key", values="dv_total_kms")
77     .reindex(index=dep_keys, columns=tof_keys)
78     .to_numpy()
79     .astype(np.float64)
80     )
81
82     return dep_times, tofs_days.astype(np.float64), grid, dep_julians, dep_index, tof_index, dep_decimals,
83         tof_decimals
84
85     def plot_porkchop_triptych(dep_times, tofs_days, dv_true, dv_pred, dv_err, tag):
86         os.makedirs("plots", exist_ok=True)
87
88         dep_mpl = mdates.date2num(dep_times.to_datetime())
89         D_dep, D_tof = np.meshgrid(dep_mpl, tofs_days, indexing="ij")
90
91         fig, axes = plt.subplots(1, 3, figsize=(18, 5), constrained_layout=True)
92
93         vmin = np.nanmin(dv_true)
94         vmax = np.nanmax(dv_true)
95         err_abs_max = np.nanmax(np.abs(dv_err))
96
97         configs = [
98             ("True Lambert V ", dv_true, (vmin, vmax), r"Total $\Delta V$ [km/s]"),
99             ("Surrogate Predicted V ", dv_pred, (vmin, vmax), r"Total $\Delta V$ [km/s]"),
100            ("Error (Pred - True)", dv_err, (-err_abs_max, err_abs_max), r"Error [km/s]"),
101        ]
102
103        for ax, (title, grid, (vmin_c, vmax_c), cbar_label) in zip(axes, configs):
104            Z = np.ma.masked_invalid(grid)
105            cs = ax.contourf(D_dep, D_tof, Z, levels=30, vmin=vmin_c, vmax=vmax_c)
106            ax.set_title(title)
107
108        try:
109            cs_lines = ax.contour(D_dep, D_tof, Z, levels=10, colors="k", linewidths=0.5)
110            ax.clabel(cs_lines, inline=True, fontsize=7, fmt="%.1f")
111        except Exception:
112            pass
113
114        ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m-%d"))
115        ax.xaxis.set_major_locator(mdates.AutoDateLocator())
116        ax.tick_params(axis="x", rotation=30)
117        ax.set_xlabel("Departure Date")
118        ax.set_ylabel("Time of Flight [days]")
119
120        cbar = fig.colorbar(cs, ax=ax)
121        cbar.set_label(cbar_label)
122
123        save_path = os.path.join("plots", f"porkchop_triptych_{tag}.png")
124        plt.savefig(save_path, dpi=300)
125        print(f"Saved 1x3 triptych: {save_path}")
126
127        def plot_error_histogram(errors, tag):
128            os.makedirs("plots", exist_ok=True)
129
130            plt.figure(figsize=(6, 4))
131            plt.hist(errors, bins=50)
132            plt.xlabel(r"Error $\Delta V_{\mathrm{pred}} - \Delta V_{\mathrm{true}}$ [km/s]")
133            plt.ylabel("Count")
134            plt.title("Surrogate V Error Histogram")
135            plt.grid(True, alpha=0.3)
136            plt.tight_layout()
137
138            hist_path = os.path.join("plots", f"porkchop_error_histogram_{tag}.png")
139            plt.savefig(hist_path, dpi=300)
140            print(f"Saved error histogram: {hist_path}")
141
142        def plot_porkchop_four_mins(
143            dep_times,
144            tofs_days,
145            dv_true_grid,
146            grid_min_true,

```

```

147     true_opt_min,
148     sur_grid_min,
149     sur_opt_min,
150     tag,
151 ):
152     os.makedirs("plots", exist_ok=True)
153
154     dep_mpl = mdates.date2num(dep_times.to_datetime())
155     D_dep, D_tof = np.meshgrid(dep_mpl, tofs_days, indexing="ij")
156     Z = np.ma.masked_invalid(dv_true_grid)
157
158     plt.figure(figsize=(10, 5))
159     cs = plt.contourf(D_dep, D_tof, Z, levels=30)
160     cbar = plt.colorbar(cs)
161     cbar.set_label(r"Total $\Delta V$ [km/s]")
162
163     ax = plt.gca()
164     ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m-%d"))
165     ax.xaxis.set_major_locator(mdates.AutoDateLocator())
166     plt.xticks(rotation=30, ha="right")
167     plt.xlabel("Departure Date")
168     plt.ylabel("Time of Flight [days]")
169
170     def pt(dep_time_tdb, tof_days):
171         return mdates.date2num(dep_time_tdb.to_datetime()), float(tof_days)
172
173     xg, yg = pt(grid_min_true["dep_time"], grid_min_true["tof_days"])
174     plt.scatter([xg], [yg], marker="*", s=180, label=f"Grid true min: {grid_min_true['dv_kms']:.3f}")
175
176     if true_opt_min is not None:
177         xo, yo = pt(true_opt_min["dep_time"], true_opt_min["tof_days"])
178         plt.scatter([xo], [yo], marker="X", s=140, label=f"True opt min: {true_opt_min['dv_kms']:.3f}")
179
180     xsg, ysg = pt(sur_grid_min["dep_time"], sur_grid_min["tof_days"])
181     plt.scatter([xsg], [ysg], marker="D", s=90, label=f"Sur grid min (pred): {sur_grid_min['dv_kms']:.3f}")
182
183     xso, yso = pt(sur_opt_min["dep_time"], sur_opt_min["tof_days"])
184     plt.scatter([xso], [yso], marker="o", s=90, label=f"Sur opt min (pred): {sur_opt_min['dv_pred_kms']:.3f}")
185
186     plt.title("Porkchop minima overlay")
187     plt.legend(loc="best")
188     plt.tight_layout()
189
190     out_path = os.path.join("plots", f"porkchop_four_mins_{tag}.png")
191     plt.savefig(out_path, dpi=300)
192     print(f"Saved four-mins plot: {out_path}")
193
194     def load_norm_stats(path):
195         with open(path, "r") as f:
196             norm_info = json.load(f)
197             X_mean = np.array(norm_info["X_mean"], dtype=np.float32)
198             X_std = np.array(norm_info["X_std"], dtype=np.float32)
199             return X_mean, X_std
200
201
202     def load_model(path, device):
203         ckpt = torch.load(path, map_location=device)
204         in_dim = int(ckpt.get("in_dim", 2))
205         hidden_dim = int(ckpt.get("hidden_dim", 128))
206         dropout_p = float(ckpt.get("dropout_p", 0.1))
207         y_mean = float(ckpt.get("y_mean", 0.0))
208         y_std = float(ckpt.get("y_std", 1.0))
209
210         model = DVSSurrogate(in_dim=in_dim, hidden_dim=hidden_dim, dropout_p=dropout_p)
211         model.load_state_dict(ckpt["model_state_dict"])
212         model.to(device)
213         model.eval()
214
215         return model, in_dim, hidden_dim, dropout_p, y_mean, y_std
216
217
218     def predict_surrogate_kms(model, dep_jd, tof_days, X_mean, X_std, y_mean, y_std, device):
219         x = np.array([[dep_jd, tof_days]], dtype=np.float32)

```

```

220 x_norm = (x - X_mean) / X_std
221 x_t = torch.from_numpy(x_norm).to(device)
222 with torch.no_grad():
223     y_norm = float(model(x_t).detach().cpu().numpy()[0, 0])
224     return float(y_norm * y_std + y_mean)
225
226
227 def build_surrogate_grid(
228     df_clean,
229     dv_true_grid,
230     dep_index,
231     tof_index,
232     model,
233     X_mean,
234     X_std,
235     y_mean,
236     y_std,
237     device,
238     dep_decimals=9,
239     tof_decimals=6,
240 ):
241     dv_pred_grid = np.full_like(dv_true_grid, np.nan, dtype=np.float64)
242
243     dep_vals64 = df_clean["dep_julian"].values.astype(np.float64)
244     tof_vals64 = df_clean["tof_days"].values.astype(np.float64)
245
246     X = np.stack([dep_vals64, tof_vals64], axis=1).astype(np.float32)
247     X_norm = (X - X_mean.reshape(1, -1)) / X_std.reshape(1, -1)
248
249     X_t = torch.from_numpy(X_norm).to(device)
250     with torch.no_grad():
251         y_norm = model(X_t).detach().cpu().numpy().reshape(-1)
252
253     y_pred = y_norm * y_std + y_mean
254
255     dep_keys = np.round(dep_vals64, dep_decimals)
256     tof_keys = np.round(tof_vals64, tof_decimals)
257
258     for k in range(len(df_clean)):
259         i = dep_index[float(dep_keys[k])]
260         j = tof_index[float(tof_keys[k])]
261         dv_pred_grid[i, j] = float(y_pred[k])
262
263     return dv_pred_grid
264
265
266 def find_grid_min_from_grid(dep_times, tofs_days, dv_grid):
267     Z = np.array(dv_grid, dtype=np.float64)
268     if not np.isfinite(Z).any():
269         raise RuntimeError("No finite values in grid.")
270     idx = np.nanargmin(Z)
271     i, j = np.unravel_index(idx, Z.shape)
272     dep_time = dep_times[i].tdb
273     tof = float(tofs_days[j])
274     arr_time = (dep_time + TimeDelta(tof * u.day)).tdb
275     dv = float(Z[i, j])
276     return {"i": int(i), "j": int(j), "dep_time": dep_time, "tof_days": tof, "arr_time": arr_time, "dv_kms": dv}
277
278
279 def find_surrogate_grid_min(dep_times, tofs_days, dv_pred_grid):
280     Z = np.array(dv_pred_grid, dtype=np.float64)
281     if not np.isfinite(Z).any():
282         raise RuntimeError("No finite values in surrogate grid.")
283     idx = np.nanargmin(Z)
284     i, j = np.unravel_index(idx, Z.shape)
285     dep_time = dep_times[i].tdb
286     tof = float(tofs_days[j])
287     arr_time = (dep_time + TimeDelta(tof * u.day)).tdb
288     dv = float(Z[i, j])
289     return {"i": int(i), "j": int(j), "dep_time": dep_time, "tof_days": tof, "arr_time": arr_time, "dv_kms": dv}
290
291

```

```

292 def run_surrogate_optimizer(
293     dep_start,
294     dep_end,
295     min_tof_days,
296     max_tof_days,
297     model,
298     X_mean,
299     X_std,
300     y_mean,
301     y_std,
302     device,
303     method="de",
304     seed=2,
305 ):
306     if not _HAS SCIPY:
307         raise RuntimeError("SciPy is required for surrogate optimization")
308
309     t0 = Time(dep_start, scale="tdb").tdb
310     t1 = Time(dep_end, scale="tdb").tdb
311     dep_bounds = (float(t0.jd), float(t1.jd))
312     tof_bounds = (float(min_tof_days), float(max_tof_days))
313
314     BIG = 1e6
315     history = []
316
317     def f(x):
318         dep_jd = float(x[0])
319         tof = float(x[1])
320         if dep_jd < dep_bounds[0] or dep_jd > dep_bounds[1] or tof < tof_bounds[0] or tof > tof_bounds[1]:
321             return BIG
322         return predict_surrogate_kms(model, dep_jd, tof, X_mean, X_std, y_mean, y_std, device)
323
324     x0 = np.array([(dep_bounds[0] + dep_bounds[1]) / 2.0, (tof_bounds[0] + tof_bounds[1]) / 2.0], dtype=np.
325                   float64)
326
327     if method.lower() == "de":
328         bounds = [dep_bounds, tof_bounds]
329
330         def cb(xk, convergence=None):
331             history.append((float(xk[0]), float(xk[1]), float(f(xk))))
332             return False
333
334         res = differential_evolution(
335             f,
336             bounds=bounds,
337             seed=seed,
338             callback=cb,
339             polish=True,
340             updating="deferred",
341             workers=1,
342         )
343         xbest = res.x
344     else:
345         bounds = [dep_bounds, tof_bounds]
346
347         def cb(xk):
348             history.append((float(xk[0]), float(xk[1]), float(f(xk))))
349
350         res = minimize(
351             f,
352             x0=x0,
353             method="L-BFGS-B",
354             bounds=bounds,
355             callback=cb,
356         )
357         xbest = res.x
358
359     dep_jd_best = float(xbest[0])
360     tof_best = float(xbest[1])
361     dep_time_best = Time(dep_jd_best, format="jd", scale="tdb").tdb
362     arr_time_best = (dep_time_best + TimeDelta(tof_best * u.day)).tdb
363     dv_pred_best = float(f([dep_jd_best, tof_best]))
364
365     return {

```

```

365     "method": method,
366     "success": bool(getattr(res, "success", True)),
367     "message": str(getattr(res, "message", "")),
368     "nfev": int(getattr(res, "nfev", -1)) if getattr(res, "nfev", None) is not None else -1,
369     "dep_time": dep_time_best,
370     "tof_days": tof_best,
371     "arr_time": arr_time_best,
372     "dv_pred_kms": dv_pred_best,
373     "history": history,
374 }
375
376
377 def evaluate_surrogate(
378     dep_start,
379     dep_end,
380     min_tof_days,
381     max_tof_days,
382     clean_csv_path=CLEAN_CSV_PATH,
383     model_checkpoint=MODEL_CHECKPOINT,
384     norm_stats_path=NORM_STATS_PATH,
385     true_opt_result=None,
386     dv_cutoff=20.0,
387     surrogate_opt_method="de",
388     surrogate_opt_seed=2,
389     dep_decimals=9,
390     tof_decimals=6,
391 ):
392     if not os.path.exists(clean_csv_path):
393         raise FileNotFoundError(clean_csv_path)
394     if not os.path.exists(model_checkpoint):
395         raise FileNotFoundError(model_checkpoint)
396     if not os.path.exists(norm_stats_path):
397         raise FileNotFoundError(norm_stats_path)
398
399     tag = _window_tag(dep_start, dep_end, min_tof_days, max_tof_days)
400
401     df_clean = pd.read_csv(clean_csv_path)
402
403     (
404         dep_times,
405         tofs_days,
406         dv_true_grid,
407         _,
408         dep_index,
409         tof_index,
410         dep_decimals_used,
411         tof_decimals_used,
412     ) = rebuild_grid_from_df(df_clean, dep_decimals=dep_decimals, tof_decimals=tof_decimals)
413
414     X_mean, X_std = load_norm_stats(norm_stats_path)
415
416     device = "cuda" if torch.cuda.is_available() else "cpu"
417     model, in_dim, hidden_dim, dropout_p, y_mean, y_std = load_model(model_checkpoint, device)
418
419     dv_pred_grid = build_surrogate_grid(
420         df_clean,
421         dv_true_grid,
422         dep_index,
423         tof_index,
424         model,
425         X_mean,
426         X_std,
427         y_mean,
428         y_std,
429         device,
430         dep_decimals=dep_decimals_used,
431         tof_decimals=tof_decimals_used,
432     )
433
434     dv_err_grid = dv_pred_grid - dv_true_grid
435
436     plot_porkchop_triptych(dep_times, tofs_days, dv_true_grid, dv_pred_grid, dv_err_grid, tag)
437
438     valid_mask = np.isfinite(dv_true_grid) & np.isfinite(dv_pred_grid)

```

```

439 errors = dv_err_grid[valid_mask]
440 plot_error_histogram(errors, tag)
441
442 mae = float(np.mean(np.abs(errors)))
443 rmse = float(np.sqrt(np.mean(errors ** 2)))
444
445 grid_min_true = find_grid_min_from_grid(dep_times, tofs_days, dv_true_grid)
446 sur_grid_min = find_surrogate_grid_min(dep_times, tofs_days, dv_pred_grid)
447
448 sur_opt = run_surrogate_optimizer(
449     dep_start,
450     dep_end,
451     min_tof_days,
452     max_tof_days,
453     model,
454     X_mean,
455     X_std,
456     y_mean,
457     y_std,
458     device,
459     method=surrogate_opt_method,
460     seed=surrogate_opt_seed,
461 )
462
463 dv_true_at_sur = float(
464     delta_v_objective(
465         sur_opt["dep_time"],
466         sur_opt["tof_days"],
467         dv_cutoff=dv_cutoff,
468         return_components=False,
469     )
470 )
471
472 dv_true_at_sur_grid = float(
473     delta_v_objective(
474         sur_grid_min["dep_time"],
475         sur_grid_min["tof_days"],
476         dv_cutoff=dv_cutoff,
477         return_components=False,
478     )
479 )
480
481 plot_porkchop_four_mins(
482     dep_times,
483     tofs_days,
484     dv_true_grid,
485     grid_min_true=grid_min_true,
486     true_opt_min=true_opt_result,
487     sur_grid_min=sur_grid_min,
488     sur_opt_min=sur_opt,
489     tag=tag,
490 )
491
492 print("SURROGATE EVAL")
493 print(f"mae_kms {mae:.6f}")
494 print(f"rmse_kms {rmse:.6f}")
495
496 print("GRID TRUE MIN")
497 print(grid_min_true["dep_time"].iso)
498 print(f"{grid_min_true['tof_days']:.6f}")
499 print(grid_min_true["arr_time"].iso)
500 print(f"{grid_min_true['dv_kms']:.6f}")
501
502 if true_opt_result is not None:
503     print("TRUE OBJECTIVE OPT MIN")
504     print(true_opt_result["dep_time"].iso)
505     print(f"{true_opt_result['tof_days']:.6f}")
506     print(true_opt_result["arr_time"].iso)
507     print(f"{true_opt_result['dv_kms']:.6f}")
508     print(len(true_opt_result.get("history", [])))
509
510 print("SURROGATE GRID MIN")
511 print(sur_grid_min["dep_time"].iso)
512 print(f"{sur_grid_min['tof_days']:.6f}")

```

```

513 print(sur_grid_min["arr_time"].iso)
514 print(f"{sur_grid_min['dv_kms']:.6f}")
515 print(f"{dv_true_at_sur_grid:.6f}")
516
517 print("SURROGATE OPT MIN")
518 print(sur_opt["dep_time"].iso)
519 print(f"{sur_opt['tof_days']:.6f}")
520 print(sur_opt["arr_time"].iso)
521 print(f"{sur_opt['dv_pred_kms']:.6f}")
522 print(f"{dv_true_at_sur:.6f}")
523 print(len(sur_opt.get("history", [])))
524
525 print("SUMMARY")
526 print(f"grid_true_dv_kms {grid_min_true['dv_kms']:.6f}")
527 if true_opt_result is not None:
528     print(f"trueopt_true_dv_kms {true_opt_result['dv_kms']:.6f}")
529     print(f"surggrid_pred_dv_kms {sur_grid_min['dv_kms']:.6f}")
530     print(f"surggrid_true_dv_kms {dv_true_at_sur_grid:.6f}")
531     print(f"suropt_pred_dv_kms {sur_opt['dv_pred_kms']:.6f}")
532     print(f"suropt_true_dv_kms {dv_true_at_sur:.6f}")
533
534 return {
535     "mae_kms": mae,
536     "rmse_kms": rmse,
537     "grid_min_true": grid_min_true,
538     "true_opt_result": true_opt_result,
539     "sur_grid_min": sur_grid_min,
540     "sur_grid_min_true_dv": dv_true_at_sur_grid,
541     "sur_opt": sur_opt,
542     "sur_opt_true_dv": dv_true_at_sur,
543     "device": device,
544     "model_meta": {
545         "in_dim": in_dim,
546         "hidden_dim": hidden_dim,
547         "dropout_p": dropout_p,
548         "y_mean": y_mean,
549         "y_std": y_std,
550     },
551     "rounding": {"dep_decimals": dep_decimals_used, "tof_decimals": tof_decimals_used},
552     "tag": tag,
553 }
554
555
556 if __name__ == "__main__":
557     evaluate_surrogate(
558         dep_start="2030-01-01",
559         dep_end="2032-01-01",
560         min_tof_days=120.0,
561         max_tof_days=360.0,
562         clean_csv_path=CLEAN_CSV_PATH,
563         model_checkpoint=MODEL_CHECKPOINT,
564         norm_stats_path=NORM_STATS_PATH,
565         true_opt_result=None,
566         dv_cutoff=20.0,
567         surrogate_opt_method="de",
568         surrogate_opt_seed=2,
569         dep_decimals=9,
570         tof_decimals=6,
571     )

```

Listing 4. Evaluate Data